

CS504

Software Engineering – I

COMPLETE STUDY GUIDE

Lectures 19 – 45

Prepared for:	Muhammad Haseeb Khan
University:	Virtual University of Pakistan
Subject Code:	CS504
Coverage:	Lectures 19 to 45 (Complete)
Type:	Comprehensive Notes MCQs Q&A; Viva

■ Exam Tip

This guide covers every concept, definition, example, MCQ, Short Questions, Long Questions, and Viva from Lectures 19–45 to help you achieve A+ grade.

- _____

Final Chapter — 100 Definitions · 200 MCQs · Short/Long Answers · Mind Map

Lecture 19: Identify Structures – Object Modeling

Learning Objectives

- Identify Gen-Spec (Generalization–Specialization) structures in OO design.
- Identify Whole-Part (Composition/Aggregation) structures.
- Assign responsibilities (attributes, services, associations) to objects.
- Build a complete Object Model for a real-world system.

Introduction

Lecture 19 continues the object-oriented analysis and design case study of "Connie's Convenience Store." We learn how to identify inheritance (Gen-Spec) and composition (Whole-Part) hierarchies, and then assign What I Know, What I Do, and Who I Know responsibilities to each object.

1. Gen-Spec (Generalization–Specialization) Structures

Definition: Gen-Spec Structure

A hierarchy in which a general (parent) class is specialized into more specific (child) classes. In UML this is represented by inheritance. The child "IS-A" kind of the parent.

Examples from Connie's Store:

General Class	Specialized Classes
Payment	Cash Payment, Authorized Payment → Check, Card/Charge
Price	Promotional Price
Sale	Sale (positive amount), Return (negative amount)
Store	Could be specialized into different kinds of outlets (future)

Why Gen-Spec? It removes duplicated attributes/services. Common data lives in the parent class; children only store what is different. This follows the DRY principle (Don't Repeat Yourself).

■ Exam Tip

Exam Tip: In a Gen-Spec hierarchy the child IS-A parent. Cash Payment IS-A Payment. Promotional Price IS-A Price.

2. Whole-Part Structures

Definition: Whole-Part Structure

A hierarchy in which one object (the Whole) is composed of other objects (the Parts). The Whole "HAS-A" or "CONTAINS" the Part. This represents Aggregation or Composition in UML.

Examples from Connie's Store:

- Store (Whole) → Cashiers, Registers, Items (Parts)
- Register (Whole) → Cash Drawer (Part)
- Sale (Whole) → Sales Line Items (Parts)

■ Exam Tip

Key distinction: A child in Gen-Spec IS a kind of the parent. A Part in Whole-Part IS CONTAINED IN the Whole.

3. Establishing Responsibilities – "Who I Know" Rules

Once structures are identified, we assign responsibilities. The "Who I Know" rule of thumb says:

Rule	Example
An actor knows about its participants	Person knows about Cashier
A transaction knows about its participants	Session knows Register & Cashier
A transaction contains transaction line items	Sale contains Sales Line Items
A transaction knows its sub-transactions	Sale knows about Payments
A place knows about its transactions	Store knows about Sessions
A place knows about descriptive objects	Store knows Tax Categories
A container knows about its contents	Store knows Cashiers, Items, Registers

4. Define Attributes, Services, and Links

For each object class we define: (a) What I Know = Attributes, (b) What I Do = Services/Methods, (c) Who I Know = Associations.

Key Classes and Their Details:

Class	Attributes	Key Services
Cashier	number, password, auth-level	isAuthorized(), assessPerformance()
Store	name	getItemForUPC(), getCashierForNumber()
Item	number, description, UPCs, prices, taxable	getItemForUPC(), howMuchForQty()
Register	number	howMuchOverInterval(), howManyOverInterval()
Cash Drawer	balance, position, state	open()
Sale	date, time	calcSubtotal(), calcTotal(), calcTax(), commit()
Sale Line Item	quantity, tax status	calculateSubTotal()
Payment	amount paid	(inherited by sub-types)
Session	start/end date & time	howMuchCollected(), howManySales()

5. Payment Hierarchy

Payment types share some attributes but differ. Gen-Spec hierarchy:

- Payment (super): amount paid
- ■■■■ Cash Payment: cash tendered
- ■■■■ Authorized Payment (super)
- ■■■■ Check: bank, account number, auth code
- ■■■■ Card/Charge: card type, card number, expiry, auth code

■ Exam Tip

Common attributes (bank account fields vs card fields) go in the specialized class. Shared attribute "amount paid" stays in Payment.

Important Keywords

Term	Meaning
Gen-Spec	Generalization-Specialization – IS-A inheritance hierarchy
Whole-Part	HAS-A composition or aggregation hierarchy
UPC	Universal Product Code – unique product identifier
Tax Category	Classification for tax rate calculation
Authorized Payment	Check or Card; needs auth code unlike cash

Short Questions

Q: What is a Gen-Spec structure? Give an example from Connie's Store.

A Gen-Spec (Generalization-Specialization) structure is an IS-A inheritance hierarchy where a general parent class is refined into specialized child classes. Example: Payment → Cash Payment, Authorized Payment → Check / Card.

Q: What is a Whole-Part structure?

A Whole-Part structure shows HAS-A or CONTAINS relationships. The Whole object is composed of Part objects. Example: Store CONTAINS Cashiers, Registers, and Items.

Q: What are the three things every object must define?

"What I Know" (attributes), "What I Do" (services/methods), and "Who I Know" (associations to other objects).

Q: Why is Payment split into a Gen-Spec hierarchy?

Cash, Check, and Card payments share an amount but have different extra attributes. Gen-Spec avoids repeating common attributes and lets each subclass store only what is unique.

Q: Name any three rules of "Who I Know".

1) An actor knows its participants (Person knows Cashier). 2) A transaction knows its participants (Session knows Register). 3) A container knows its contents (Store knows Cashiers).

MCQs

Q1. In Gen-Spec structure, the relationship between child and parent is:

- (A) HAS-A
- ✓ (B) IS-A
- (C) USES-A
- (D) CREATES-A

Explanation: A child IS-A kind of the parent, e.g., Cash Payment IS-A Payment.

Q2. In Whole-Part structure, a Register CONTAINS:

- (A) A Sale
- (B) A Cashier
- ✓ (C) A Cash Drawer
- (D) A Tax Category

Explanation: Register HAS-A Cash Drawer inside it.

Q3. Which rule says "A store knows about its cashiers, items, and registers"?

- (A) Actor rule
- (B) Transaction rule
- ✓ (C) Container rule

(D) Place rule

Explanation: A container knows about its contents.

Q4. What attribute is common to ALL payment types in Connie's store?

(A) Bank number

(B) Card type

✓ (C) Amount paid

(D) Auth code

Explanation: All payments share "amount paid".

Q5. Promotional Price IS-A specialization of:

(A) Sale

(B) Item

✓ (C) Price

(D) Tax Category

Explanation: Promotional Price IS-A Price.

■ Memory Trick

GEN-SPEC = IS-A | WHOLE-PART = HAS-A

Remember: Inheritance uses IS-A. Composition uses HAS-A. Never confuse them!

Lecture 20: Interaction Diagrams – Sequence Diagrams

Learning Objectives

- Understand what Interaction Diagrams are and why they are used.
- Read and draw Sequence Diagrams with proper notation.
- Understand lifelines, activation boxes, messages, and conditions.
- Distinguish between Sequence Diagrams and Collaboration Diagrams.

Definition: Interaction Diagram

A UML diagram that shows how objects interact by exchanging messages to accomplish a specific purpose (e.g., implementing a use case).

Purpose of Interaction Diagrams

- Model interactions between objects in a use case.
- Verify that a use case can be supported by the existing classes.
- Identify responsibilities (operations) and assign them to classes.
- Assist in understanding how a system actually works dynamically.

Types of Interaction Diagrams in UML

Type	View	Emphasis
Sequence Diagram	Time-based	Message order over time (vertical axis = time)
Collaboration Diagram	Organization-based	Object relationships and message numbering

Sequence Diagram Notation

Definition: Lifeline

The dashed vertical line hanging beneath an object box. It represents the object's existence during the interaction.

Definition: Activation Box

A thin rectangle on the lifeline showing the period during which the object is actively processing a message.

Definition: Message

A horizontal arrow from one lifeline to another representing a method call or communication.

Key notation rules:

- Objects arranged horizontally on the X-axis.
- Time flows downward on the Y-axis.
- Object syntax: [instanceName][:className], e.g., member:LibraryMember
- Messages labeled with the method name (and optionally arguments).
- Conditions shown in square brackets, e.g., [ok] before a message.
- Self-calls (an object messaging itself) are allowed.

Library Borrowing Example

Three objects: :BookCopy, book:Book, member:LibraryMember

- member sends borrow(book) to :BookCopy
- :BookCopy sends ok = mayBorrow() to itself (self-call)
- If [ok] is true, :BookCopy sends borrow(member) to book:Book
- book:Book sends setTaken(member) back

■ Exam Tip

Association between objects: obj1 can send a message to obj2 only if an association exists between them (obj2 is in obj1's scope, global scope, or they are the same object).

Short Questions

Q: What is the purpose of a lifeline in a Sequence Diagram?

A lifeline (dashed vertical line below the object box) represents the object's life during the interaction. It shows the time span over which the object participates.

Q: What is an activation box?

An activation box is a thin rectangle placed on the lifeline. It shows the period during which the object is actively executing a message (i.e., how long the processing takes).

Q: How do you show a conditional message in a Sequence Diagram?

You write the condition in square brackets before the message arrow, e.g., [ok] borrow(member). The message is only sent if the condition is true.

Q: What is the naming syntax for objects in a Sequence Diagram?

[instanceName][:className]. The instanceName is optional. Example: member:LibraryMember or just :LibraryMember.

Q: What must exist between two objects for a message to be sent?

An association (structural dependency) must exist between the two objects. The receiving object must be in the global scope, local scope, or be the same object.

MCQs

Q6. In a Sequence Diagram, time flows in which direction?

- (A) Horizontally
- (B) Diagonally
- ✓ (C) Vertically downward
- (D) Vertically upward

Explanation: Time is on the Y-axis (vertical) and increases downward.

Q7. What does an activation box represent?

- (A) The object's name
- ✓ (B) The period the object is actively processing
- (C) A condition
- (D) A message return value

Explanation: It shows how long a method is active/executing.

Q8. A message that an object sends to itself is called:

- (A) Asynchronous message
- (B) Return message
- ✓ (C) Self-call
- (D) Create message

Explanation: Self-calls are messages from an object to itself.

Q9. Which diagram provides a TIME-BASED view of object interactions?

- (A) Collaboration Diagram
- (B) State Diagram
- ✓ (C) Sequence Diagram
- (D) Class Diagram

Explanation: Sequence Diagrams emphasize time ordering of messages.

Q10. The horizontal axis of a Sequence Diagram represents:

- (A) Time
- (B) Messages
- ✓ (C) Objects
- (D) Associations

Explanation: Objects are arranged horizontally.

■ **Memory Trick**

SEQUENCE = S for Space (horizontal objects) + S for Sequential time

In Sequence Diagrams: Objects spread horizontally, time runs downward. Think "space-time grid."

Lecture 21: Message Types, Collaboration Diagrams & OO Design Quality

1. Message Types in Sequence Diagrams

Message Type	Arrow Style	Description
Synchronous	Filled arrowhead (→)	Caller waits for completion; nested flow of control; operation call.
Asynchronous	Half arrowhead (⇒)	Caller does NOT wait; used in multi-threaded apps; creates threads or sends signals.
Create	<<create>> dashed arrow	One object creates another object.
Destroy	<<destroy>> dashed arrow	One object destroys another (or itself).
Return	Dashed arrow back (←-)	Optional return value from a synchronous call.

Synchronous messages – The caller is BLOCKED until the called method returns. e.g., `doYouUnderstand()` blocks until the answer "yes" comes back.

Asynchronous messages – Caller continues immediately. Used in multi-threading, event-driven systems, and message queues.

■ Exam Tip

Guidelines: Don't show return value if obvious (e.g., `getTotal()`). Show return only when the value is referenced later (e.g., `ok = isValid()`).

■ Important Note

Avoid modeling object destruction unless memory management is critical.

2. Collaboration Diagrams

Definition: Collaboration Diagram

A UML diagram that shows how objects interact organized by their relationships (boundaries). Messages are numbered to show sequence order.

Key differences from Sequence Diagrams:

Feature	Sequence Diagram	Collaboration Diagram
Primary focus	Time ordering of messages	Object organization & relationships
Message order	Shown by vertical position	Shown by numbering (1, 2, 3...)
Best for	Seeing flow of time	Seeing static object connections
Complexity	Gets unwieldy for many objects	Can get very complex quickly

3. Evaluating OO Design Quality

We evaluate OO design using coupling and cohesion, guided by encapsulation and delegation.

Definition: Coupling

The degree of dependency between objects/modules. Less coupling = easier to maintain.

Definition: Cohesion

The degree to which the elements within a module/class belong together. High cohesion = focused, single-purpose class.

Heat Regulator Example: Distributing intelligence vs. encapsulating it:

- BAD (centralized control): HeatFlowRegulator talks to three separate objects: ActualTemp, DesiredTemp, Occupancy → HIGH coupling.
- BETTER (encapsulation): Wrap three into one Room object. Regulator talks to one object → LOWER coupling.
- BEST (delegation): Room itself provides doYouNeedHeat() method. Regulator calls one method → LOWEST coupling.

■ Exam Tip

Design principle: Minimize the number of messages in a class's public interface (protocol). Small, focused interfaces are easier to understand and maintain.

Consolidation example – instead of two methods setMinimumValue() and setMaximumValue(), combine into one: setLimits(min, max). This reduces protocol size.

Short Questions**Q: What is the difference between synchronous and asynchronous messages?**

Synchronous: caller waits (blocks) until method completes. Asynchronous: caller does not wait; continues immediately. Async used in multi-threaded apps.

Q: How is message order shown in a Collaboration Diagram?

By numbering: messages are labeled 1, 2, 3, etc. showing the sequence in which they are sent.

Q: What does "distributed control" mean in OO design?

Intelligence is spread across separate objects (e.g., three separate temp/occupancy objects). This increases coupling because the controller must communicate with many objects.

Q: How can we reduce coupling in OO design?

By encapsulating related objects into a single cohesive class and using delegation (asking the encapsulated object to compute results instead of asking for raw data).

Q: What is the "golden principle" for Sequence Diagrams complexity?

Keep it small and simple. If control logic is complex, draw separate diagrams (e.g., flowcharts) rather than cluttering the Sequence Diagram.

MCQs**Q11. An asynchronous message is denoted by:**

- (A) Filled arrowhead
- ✓ (B) Half arrowhead
- (C) Dashed line
- (D) Solid box

Explanation: Half arrowhead = async (caller not blocked).

Q12. In Collaboration Diagrams, message sequence is shown by:

(A) Position (top to bottom)

✓ (B) Numbering (1,2,3)

(C) Color

(D) Arrow thickness

Explanation: Messages are numbered to show order.

Q13. High cohesion means:

(A) Many dependencies

✓ (B) A focused single-purpose class

(C) Distributed intelligence

(D) Large public interface

Explanation: High cohesion = elements inside class are closely related.

Q14. Which is the BEST design for the heat regulator?

(A) Regulator calls 3 objects

(B) Regulator calls 1 Room object for raw data

✓ (C) Regulator calls Room.doYouNeedHeat()

(D) No encapsulation

Explanation: Delegation to Room reduces coupling the most.

Q15. Reducing the number of methods in a class's public interface leads to:

(A) Lower cohesion

(B) Higher coupling

✓ (C) Lower coupling

(D) More classes

Explanation: Smaller interfaces reduce coupling.

■ Memory Trick

LESS coupling + MORE cohesion = BETTER OO design

Think of a well-organized toolbox: tools grouped by type (cohesion), and you only need one toolbox (low coupling).

Lecture 22: Software Architecture – Introduction & Importance

Learning Objectives

- Define software architecture from multiple authoritative sources.
- Explain why architecture is important (three main reasons).
- Understand architectural attributes: Performance, Security, Safety, Availability, Maintainability.
- Describe the architectural design process (structuring, control modeling, decomposition).

Definition: Software Architecture

The structure(s) of a system, comprising software components, the externally visible properties of those components, and the relationships among them. (Bass, Clements, Kazman 1997)

Architecture Definitions from Key Sources

Source	Definition
UML 1.3	Organizational structure of a system; recursively decomposable into parts that interact through interfaces
Garlan & Perry (IEEE TSE 1995)	Structure of components, their interrelationships, principles and guidelines governing design and evolution
Shaw & Garlan	Defines system in terms of computational components and interactions among them (clients, servers, etc.)
IEEE Glossary	Process of defining hardware/software components and interfaces to establish the development framework

Historical background: Edsger Dijkstra (1968) first advocated concern for software structure → layered OS. David Parnas contributed information-hiding modules and program families.

Why Is Architecture Important? (3 Reasons)

1. Mutual Communication

Architecture is a common high-level abstraction that all stakeholders (developers, managers, customers, testers) can use as a basis for discussion, decision-making, and understanding. Without it, large systems cannot be managed intellectually.

2. Early Design Decisions

Architecture embodies the earliest design decisions. These early decisions carry weight disproportionate to their size. The architecture determines the work breakdown structure (WBS), team organization, schedules, budgets, and inter-team communication. Once frozen, changing architecture becomes very expensive.

3. Reusable Abstraction

Architecture provides a small, intellectually graspable model that can be transferred across systems with similar requirements. This enables large-scale reuse through product lines (a family of closely related architectures).

■ Exam Tip

Boehm's Rule: "If a project has not achieved a system architecture, including its rationale, the project should not proceed to full-scale system development."

Architectural Attributes (Non-Functional Requirements)

Attribute	Guideline
-----------	-----------

Performance	Localize operations; minimize subsystem communication; use self-contained modules.
Security	Use layered architecture; place critical assets in inner layers.
Safety	Isolate safety-critical components.
Availability	Build redundancy into the architecture; use redundant components.
Maintainability	Use fine-grain, self-contained components; keep it simple.

Architectural Design Process

Architecture design is creative and iterative. Three key activities:

- **System Structuring:** Decompose system into interacting sub-systems; identify communication between sub-systems. Output: block diagram.
- **Control Modeling:** Establish the control relationships between different parts of the system.
- **Modular Decomposition:** Decompose identified sub-systems further into modules.

Definition: Sub-system

A system in its own right; operates independently of other subsystems' services.

Definition: Module

A system component that provides services to other components; not normally considered a separate system.

Short Questions

Q: Give two definitions of software architecture from different sources.

1) UML 1.3: Organizational structure of a system, recursively decomposable into parts interacting through interfaces. 2) Bass et al.: Structure(s) of a system comprising software components, their externally visible properties, and relationships.

Q: What are the three reasons why architecture is important?

1) Mutual Communication – gives stakeholders a common abstraction. 2) Early Design Decisions – architectures carry disproportionate long-term weight. 3) Reusable Abstraction – architecture models can be reused across similar systems.

Q: How does architecture address performance?

By localizing operations to minimize subsystem communication and using self-contained modules to reduce inter-module messages.

Q: What is the difference between a sub-system and a module?

A sub-system operates independently; it is a system in its own right. A module provides services to other components but is not a separate system on its own.

Q: Who first introduced the concept of layered software structure?

Edsger Dijkstra in 1968 when writing about an operating system. He proposed a layered structure where programs in one layer communicate only with programs in adjoining layers.

MCQs

Q16. According to Barry Boehm, a project should NOT proceed to full-scale development unless:

(A) Code is written

✓ (B) Architecture with rationale is achieved

(C) Testing is done

(D) Budget is approved

Explanation: Boehm requires architecture + rationale before full-scale development.

Q17. Architecture ensures "mutual communication" by:

(A) Writing code

✓ (B) Providing a common high-level abstraction

(C) Freezing requirements

(D) Hiring more developers

Explanation: It is a shared language for all stakeholders.

Q18. Which attribute is addressed by putting critical assets in INNER layers?

(A) Performance

(B) Maintainability

✓ (C) Security

(D) Availability

Explanation: Security: inner layers protect critical assets.

Q19. Redundant components in an architecture primarily address:

(A) Performance

(B) Security

(C) Safety

✓ (D) Availability

Explanation: Redundancy ensures system stays available when components fail.

Q20. The output of system structuring is typically:

(A) Source code

(B) Test cases

✓ (C) A block diagram

(D) A database schema

Explanation: Block diagram presents overview of system structure.

■ Memory Trick

3 Reasons for Architecture: M-E-R

M = Mutual Communication, E = Early Decisions, R = Reusable Abstraction. "MER architecture" – like a ship's captain (MERchant) needs a plan before sailing.

Lecture 23: Architectural Views – Krutchen 4+1 Model & Styles

Perry-Wolfe Formula for Software Architecture

Formula

Software Architecture = {Elements, Forms, Rationale/Constraints} Elements: Data elements, Processing elements, Connecting elements. Forms: How elements are organized. Rationale: Justification for architectural decisions.

Krutchen's 4+1 Architectural View Model

Definition: Krutchen's 4+1 Model

A model proposing that software architecture should be described using FIVE views: Logical, Process, Physical, Development, and Use Case (scenarios). Each view addresses different stakeholders.

View	What It Covers	Primary Users
Logical View	Object model of the design; classes and relationships	End users, Analysts
Process View	Concurrency and synchronization; parallel processes	System Integrators, Performance Engineers
Physical View (Deployment)	Mapping of software onto hardware; distribution	System/Hardware Engineers
Development View (Implementation)	Static organization; files, directories	Programmers, Configuration Managers
Use Case View (Scenarios)	Selected use cases that validate architecture	All stakeholders

Clements Modified Version – 5 Views

View	Components	Key Uses
Functional View	Functions, key abstractions, domain elements	Functionality, modifiability, reusability
Code View	Classes, objects, procedures, subsystems, modules	Maintainability, portability
Development View	Files, directories	Maintenance, configuration management
Physical View	CPUs, sensors, storage	System delivery, performance, security
Concurrency View	Processes, threads	Performance, availability, parallelism

Scenarios tie all views together. They describe sequences of responsibilities and change cases. They validate the system and communicate the design to stakeholders.

Hierarchical Views

- Functional View: contains sub-functions.
- Development View: directories → files.
- Code View: modules → sub-modules, systems → sub-systems.
- Concurrency View: processes → threads.
- Physical View: clusters → computers → processors.

■ Exam Tip

Every view is potentially hierarchical! A good architecture should have the right number of hierarchical levels to support desired analyses.

Architectural Styles

Definition: Architectural Style

A pattern of architectural model that describes a system category, including: set of components, set of connectors, constraints, and semantic models.

Common architectural styles:

- Data-Centered (Repository) Architecture
- Client-Server Architecture and variations
- Layered Architecture
- Reference Architecture (e.g., OSI model)

Short Questions

Q: State the Perry-Wolfe formula for software architecture.

Software Architecture = {Elements, Forms, Rationale/Constraints}. Elements = data, processing, connecting. Forms = organization. Rationale = justification of decisions.

Q: List the five views in Krutchen's 4+1 model.

1) Logical View (object model). 2) Process View (concurrency). 3) Physical View (hardware mapping). 4) Development View (files/directories). 5) Use Case/Scenario View (validation).

Q: What is the purpose of Scenarios in the 4+1 model?

Scenarios (use cases) tie all four views together into an integrated system. They validate architecture against functional requirements and help understand dynamic behavior and design limits.

Q: Who are the primary users of the Physical View?

Hardware engineers and system engineers. The Physical View shows processors, sensors, storage, and network components.

Q: What is an architectural style?

A pattern or template for architecture that includes a set of components, connectors, constraints, and semantic models. It provides a reusable solution to recurring architectural problems.

MCQs

Q21. In Krutchen's model, the Logical View captures:

- (A) Concurrency
- ✓ (B) Object model of the design
- (C) Hardware mapping
- (D) Files and directories

Explanation: Logical View = object model (classes/relationships).

Q22. The view that documents files and directories is:

- (A) Physical View

(B) Logical View

✓ (C) Development View

(D) Process View

Explanation: Development View = static organization of software (files, directories).

Q23. Scenarios (use cases) serve what purpose in Krutchen's model?

(A) They replace all other views

✓ (B) They tie all views together

(C) They show only physical deployment

(D) They list algorithms

Explanation: Scenarios validate and integrate all four main views.

Q24. The Concurrency View is primarily used by:

(A) End users

(B) Hardware engineers

✓ (C) Performance engineers and integrators

(D) Customers

Explanation: Concurrency View for performance engineers, integrators, testers.

Q25. Every architectural view is potentially:

(A) Flat

✓ (B) Hierarchical

(C) Static only

(D) Textual

Explanation: Views can be nested/hierarchical (e.g., functions → sub-functions).

■ Memory Trick

4+1 Views = L-P-P-D+S

Logical, Process, Physical, Development + Scenarios. Remember: "Let People Please Developers with Scenarios."

Lecture 24: Architectural Styles – Repository, Client-Server, Layered

1. Data-Centered (Repository) Architecture

Definition: Repository Model

All sub-systems share a central database (repository). Sub-systems access shared data directly rather than communicating with each other directly.

Advantages	Disadvantages
Efficient sharing of large amounts of data	Sub-systems must agree on one data model (compromise)
Sub-systems need not concern themselves with data production	Data evolution is difficult and expensive
Centralised management (backup, security)	Little scope for specific management policies
Global view; sharing model published as repository schema	Difficult to distribute efficiently

2. Client-Server Architecture

Definition: Client-Server Model

A distributed system model where application is split into SERVERS (provide specific services) and CLIENTS (use those services). They communicate over a network. Clients know servers; servers do NOT need to know all clients.

Key characteristics:

- The system is organized as a set of stand-alone servers and clients.
- Clients and servers are LOGICAL processes (not necessarily separate physical machines).
- Mapping of processes to processors is not always 1:1.
- Middleware provides connection protocols between clients and servers.

Representative Server Types:

Server Type	What Happens
File Server	Client requests records from a file; server transmits records over network.
Database Server	Client sends SQL queries; server processes and returns results.
Transaction Server	Client invokes remote procedures; server executes and returns results.
Groupware Server	Provides communication apps: text, images, bulletin boards, video.
Advantages	Disadvantages
Effective use of networked systems; cheaper hardware	No standard for data sharing between sub-systems
Easy to add new servers or upgrade existing ones	Redundant management needed at each server
Straightforward data distribution	No central register of names/services

3. Client-Server Configurations

Thin Client Model: All processing at the server side. Client only handles display (GUI). Originally used to migrate legacy systems. Disadvantage: heavy load on server AND network.

Fat Client Model: More processing delegated to client machine. Application logic partially runs on client. Advantage: uses client's computing power. Disadvantage: complex management; every client machine needs application installation.

4. Layered Architecture

Definition: Layered Architecture

Architecture divided into multiple layers where each layer provides services to the layer above it and uses services from the layer below. Inner layers are closer to hardware; outer layers closer to users.

- Example: Operating System with User Interface → Application → Utility → Core layers.
- Each layer isolates outer layers from inner complexity.
- If inner layer implementation changes but interface stays same, outer layer is unaffected → great portability.
- OSI 7-layer model is a famous reference architecture based on this style.

■ Exam Tip

Layered architecture dramatically increases portability. Outer layers only need to know the interface, not the implementation of the inner layers.

Short Questions

Q: What is the main advantage of the Repository model?

Efficient sharing of large amounts of data with centralized management (backup, security). Sub-systems do not need to manage data production themselves.

Q: What is the main disadvantage of the Client-Server model?

No standard way of sharing data between sub-systems, so data interchange may be inefficient. Also no central register of names and services.

Q: What is the difference between Thin and Fat Client models?

Thin Client: all processing at server; client only displays. Heavy server/network load. Fat Client: processing distributed to client machine; uses client power but complex to manage (every client needs app installation).

Q: Name two types of servers in a Client-Server system.

File Server (transfers file records) and Database Server (processes SQL queries and returns results).

Q: How does Layered architecture improve portability?

Each layer only needs to know the interface of the layer below, not its implementation. If an inner layer's implementation changes but the interface stays the same, outer layers are unaffected.

MCQs

Q26. In the Repository model, all sub-systems share:

- (A) Direct network connections
- ✓ (B) A central database
- (C) Individual private databases
- (D) A messaging queue

Explanation: Repository = central shared database for all subsystems.

Q27. In Client-Server model, servers need to know:

- (A) All client names
- (B) Only specific clients
- ✓ (C) They do NOT need to know all clients
- (D) Client IP addresses

Explanation: Clients know servers; servers do NOT need to know all clients.

Q28. Fat Client model shifts processing to:

- (A) The server
- (B) The network
- ✓ (C) The client machine
- (D) The database

Explanation: Fat client = more processing on client side.

Q29. Which server type processes SQL queries?

- (A) File Server
- ✓ (B) Database Server
- (C) Transaction Server
- (D) Groupware Server

Explanation: Database Server processes SQL queries.

Q30. Layered architecture primarily improves:

- (A) Security
- ✓ (B) Portability
- (C) Availability
- (D) Performance

Explanation: Layers isolate implementation, greatly improving portability.

■ Memory Trick

Client-Server: Clients CALL, Servers SERVE

Like a restaurant: Clients (customers) ORDER from Servers (waiters/kitchen). Servers don't chase customers; customers come to servers.

Lecture 25: Zero Install, N-Tier, Pipes & Filters, Reference Architecture

1. Zero Install Architecture

Definition: Zero Install

Architecture where NO installation is required on the client side. All processing happens server-side. The client only needs a browser. Example: Web applications where web pages are hosted on a server.

Trade-off: Zero Install is essentially a return to thin-client architecture to solve fat-client installation and maintenance problems. Changes are applied on the server; all clients automatically get the updated version.

2. N-Tier Architecture

Definition: N-Tier Architecture

A generalization of client-server that distributes both data AND application across MULTIPLE server machines to enhance scalability and performance. Different servers handle different layers: web server, application server, database server.

Three-Tier Architecture (a special case of N-Tier):

Tier	Role	Example
Client Tier (Tier 1)	Presentation layer (UI)	Web browser
Application Server (Tier 2)	Business logic	Web server processing requests
Database Server (Tier 3)	Data management	SQL database

- Better performance than thin-client (logic separated from data).
- Simpler to manage than fat-client (no client-side installation).
- Highly scalable: add more application servers as demand grows.

3. Pipes and Filters (Data Flow) Architecture

Definition: Pipes and Filters

Architecture where data is processed through a series of transformations. Each processing step is a FILTER; the connection between filters is a PIPE through which data flows. Filters work independently – they do not need to know about each other.

- Filter: a processing component that transforms input data to output data.
- Pipe: a connector that passes data from one filter to the next.
- Key feature: each filter is independent of all others.
- Batch Sequential: filters arranged in a single linear sequence.
- Non-batch: filters can run in parallel with branching paths.

■ Exam Tip

Example of Pipes & Filters: Unix command pipeline. E.g., `ls | grep ".txt" | sort | head -10`. Each command is a filter; `|` is the pipe.

4. Reference Architecture

Definition: Reference Architecture

Not a physical architecture. A standard reference model defining protocols for systems built by different parties. Derived from the application domain. Acts as a standard for system evaluation and comparison. Example: OSI 7-Layer Model for networking.

OSI Reference Model (7 Layers): Physical | Data Link | Network | Transport | Session | Presentation | Application

■ Exam Tip

The Internet works because ALL heterogeneous systems use the same reference architecture (TCP/IP model). This enables any device to communicate with any other.

5. Partitioning the Architecture

Definition: Partitioning

Distributing responsibilities across sub-systems to get a maintainable, testable, and extensible software system.

Horizontal Partitioning: Separate branches of module hierarchy for each major function. Control modules coordinate communication.

Vertical Partitioning (Factoring): Decision-making modules at the TOP; worker modules at the BOTTOM. Decision makers are separate from implementers.

6. Analyzing Architecture Design

Steps for architectural analysis:

- 1. Collect scenarios.
- 2. Elicit requirements, constraints, and environment description.
- 3. Describe architectural styles/patterns chosen to address scenarios (module view, process view, data flow view).
- 4. Evaluate quality attributes in isolation.
- 5. Identify sensitivity of quality attributes to architectural attributes.
- 6. Critique candidate architectures using sensitivity analysis.

Short Questions

Q: What problem does Zero Install solve?

Zero Install solves the fat-client maintenance problem. In fat-client, every client machine needs the application installed. Zero Install keeps everything server-side; clients (browsers) get updates automatically.

Q: What are the three tiers in a Three-Tier architecture?

1) Client tier (presentation/UI). 2) Application Server tier (business logic). 3) Database Server tier (data management/storage).

Q: What is a Filter in Pipes & Filters architecture?

A filter is an independent processing component that receives input data, transforms it, and outputs the result. It does not need to know about neighboring filters.

Q: What is the OSI model? Why is it a reference architecture?

The OSI (Open Systems Interconnection) model is a 7-layer standard for network communication. It is a reference architecture because it is a standard that all systems must follow, enabling heterogeneous systems to communicate.

Q: What is the difference between horizontal and vertical partitioning?

Horizontal partitioning: separate branches per major function with control modules. Vertical partitioning (factoring): decision-makers at top, workers at bottom.

Master Revision – Lectures 19–23

All Key Definitions

Term	Quick Definition
Gen-Spec	IS-A inheritance hierarchy; child is a kind of parent
Whole-Part	HAS-A composition; whole contains parts
Lifeline	Dashed vertical line in Sequence Diagram = object's life
Activation Box	Thin rectangle on lifeline = period of active processing
Synchronous Message	Caller waits (blocks) for completion
Asynchronous Message	Caller does NOT wait; continues immediately
Software Architecture	Structure of a system: components + properties + relationships
Repository Model	Central shared database accessed by all subsystems
Client-Server	Distributed model: clients use services provided by servers
Thin Client	All processing at server; client only displays
Fat Client	Processing distributed to client machine
N-Tier	Multiple server tiers for scalability
Pipes & Filters	Data processed through independent filter components via pipes
Reference Architecture	Standard non-physical model (e.g., OSI) for protocol definition
Cohesion	Degree elements within a class belong together (HIGH = good)
Coupling	Degree of dependency between classes (LOW = good)

Most Expected MCQs from Lectures 19–23

Q31. Which architecture style uses a central database shared by all sub-systems?

- (A) Client-Server
- (B) Layered
- ✓ (C) Repository
- (D) Pipes and Filters

Explanation: Repository model uses central shared database.

Q32. What does a lifeline represent in a Sequence Diagram?

- (A) A message
- ✓ (B) Object existence during interaction
- (C) A condition
- (D) An activation period

Explanation: Lifeline = dashed vertical line = object's life in interaction.

Q33. Which architectural view captures concurrency and synchronization?

- (A) Logical View
- ✓ (B) Process View
- (C) Physical View
- (D) Development View

Explanation: Process View = concurrency and synchronization.

Q34. In Pipes and Filters, each filter:

- (A) Knows all other filters
- ✓ (B) Is independent of other filters
- (C) Shares a database
- (D) Calls a server

Explanation: Filters are independent; they only know about their input/output.

Q35. Three-Tier architecture improves on fat-client by:

- (A) Removing all servers
- (B) Keeping all logic on client
- ✓ (C) Separating into client, app server, DB server
- (D) Using zero installation

Explanation: 3-tier distributes layers to separate servers.

■ Memory Trick

N-Tier: More Tiers = More Scalability

Think of a cake: each layer is a tier. More layers = bigger cake = more scalability. But too many layers = harder to cut!

Lecture 26: Introduction to Design Patterns

Definition: Design Pattern

"Description of communicating objects and classes that are customized to solve a general design problem in a particular context." – GoF (Gang of Four). A pattern is a reusable solution to a recurring problem.

Historical Background

- Originated from architect Christopher Alexander's books (1977): A Pattern Language.
- Re-emerged in software at OOPSLA 1987 (Booch, Helm, Gamma, Beck).
- Formalized in 1995: "Design Patterns: Elements of Reusable Object-Oriented Software" by Gamma, Helm, Johnson, Vlissides (GoF – Gang of Four).
- Smalltalk used Observer Pattern as the basis of its MVC (Model-View-Controller).

■ Exam Tip

Related concepts: STL (C++ library of reusable components), Algorithms in data structures, Anti-patterns (common mistakes), OO Frameworks.

GoF Design Pattern Format (10 Elements)

Element	Description
Name	Short meaningful identifier; works as idiom
Problem	Statement of intent; goals and objectives in a given context
Context	Preconditions under which the pattern applies; resulting state
Forces	Relevant constraints and their interactions
Solution	Static and dynamic relationships; structural description with diagrams
Examples	Sample applications showing specific context and how pattern is applied
Resulting Context	State after pattern is applied; consequences (good and bad)
Rationale	Justification; how pattern resolves forces to achieve goals
Related Patterns	Static and dynamic relationships with other patterns
Known Uses	Proof that this is a proven solution to a recurring problem

Classifications of Design Patterns

Category	Concern	Examples
Creational	How objects are created/instantiated; make system independent of creation mechanism	Abstract Factory, Factory Method, Singleton
Structural	How classes/objects compose to form larger structures	Facade, Adapter, Decorator, Composite
Behavioral	Patterns of communication/interaction between objects	Observer, Strategy, Responsibility, Iterator

Class patterns use inheritance to vary behavior. **Object patterns** use object composition (delegation) to vary behavior.

■ Exam Tip

Memory trick: C-S-B = Creational patterns Create objects, Structural patterns Structure them, Behavioral patterns define how they Behave.

Short Questions

Q: What is a design pattern? Who popularized it in software?

A design pattern is a reusable solution to a commonly occurring design problem in a particular context. Christopher Alexander introduced the concept for buildings; it was popularized in software by the Gang of Four (Gamma, Helm, Johnson, Vlissides) in 1995.

Q: What are the three categories of GoF design patterns?

1) Creational – how objects are created. 2) Structural – how objects/classes compose into larger structures. 3) Behavioral – patterns of communication and responsibility between objects.

Q: What is an Anti-pattern?

Anti-patterns are common mistakes in analysis and design that are identified to prevent design defects from entering the system.

Q: Name all 10 elements in GoF format.

Name, Problem, Context, Forces, Solution, Examples, Resulting Context, Rationale, Related Patterns, Known Uses.

Q: What is an OO Framework?

A set of cooperative classes that make up a reusable design of a system. It dictates the architecture and describes the limitations and boundaries of the system.

MCQs

Q36. The GoF book on design patterns was published in:

- (A) 1977
- (B) 1987
- ✓ (C) 1995
- (D) 2001

Explanation: Published 1995 by Gamma, Helm, Johnson, Vlissides.

Q37. Which category of patterns deals with HOW objects are created?

- (A) Structural
- (B) Behavioral
- ✓ (C) Creational
- (D) Functional

Explanation: Creational patterns abstract the instantiation process.

Q38. Observer Pattern was first used in Smalltalk as part of:

- (A) Factory Method
- ✓ (B) MVC (Model-View-Controller)
- (C) Facade Pattern
- (D) Singleton Pattern

Explanation: MVC in Smalltalk was based on Observer pattern.

Q39. An Anti-pattern identifies:

- (A) Reusable solutions
- ✓ (B) Common design mistakes
- (C) New programming languages

(D) OOP concepts

Explanation: Anti-patterns document common mistakes to avoid them.

Q40. Structural patterns are concerned with:

(A) Object creation

(B) Communication between objects

✓ **(C) Composition into larger structures**

(D) Algorithm selection

Explanation: Structural = how classes/objects compose into larger structures.

■ Memory Trick

GoF = Gang of Four = 3 Pattern Types = C, S, B

Creational (birth), Structural (body), Behavioral (behaviour). Like a person: born (C), has structure (S), and behaves (B).

Lecture 27: Observer, Singleton & Facade Patterns

1. Observer Pattern (Behavioral)

Definition: Observer Pattern

Defines a ONE-TO-MANY dependency between objects. When one object (Subject) changes state, ALL its dependants (Observers) are automatically notified and updated.

Also known as: Publish-Subscribe pattern

Classic use: Spreadsheet + Bar Chart displaying same data. When spreadsheet data changes, bar chart updates automatically without them knowing about each other.

Participant	Role
Subject	Knows its observers; provides attach/detach interface; notifies observers on state change.
Observer	Defines update() interface for objects to be notified.
ConcreteSubject	Stores state of interest; sends notification when state changes.
ConcreteObserver	Maintains reference to ConcreteSubject; implements update() to keep state consistent.

Consequences: Optimizations to enhance display performance are impractical since observers are updated regardless of what changed.

2. Singleton Pattern (Creational)

Definition: Singleton Pattern

Ensures that a class has ONLY ONE INSTANCE and provides a GLOBAL POINT OF ACCESS to it.

When to use:

- When exactly one instance of a class must exist.
- When controlling the total number of instances to be created.
- When the sole instance must be extensible by sub-classing and clients use the extended instance without modifying code.

Implementation (C++):

```
class Singleton { public: static Singleton* Instance(); protected: Singleton(); private: static Singleton* _instance; }; Singleton* Singleton::Instance() { if (_instance == 0) { _instance = new Singleton; } return _instance; }
```

■ Exam Tip

Singleton: The constructor is PROTECTED so no external code can create instances. Only the static Instance() method can create or return the single instance.

3. Facade Pattern (Structural)

Definition: Facade Pattern

Provides a UNIFIED, SIMPLIFIED INTERFACE to a complex set of interfaces in a sub-system. Makes the sub-system easier to use from the client perspective.

When to use:

- When you want to provide a simple interface to a complex sub-system.
- When there are many dependencies between clients and implementation classes.
- To decouple the system from clients and other sub-systems.
- To layer your sub-system (provide a well-defined entry point).

Participant	Role
Facade	Knows which subsystem classes handle which requests; delegates client requests to appropriate subsystem
Subsystem classes	Implement functionality; handle work assigned by Facade; have NO knowledge of the Facade.

Comparison of the Three Patterns

Pattern	Category	Intent	Key Benefit
Observer	Behavioral	Notify dependants on state change	Loose coupling between subject and observers
Singleton	Creational	Ensure single instance; global access	Controlled instance creation
Facade	Structural	Simplify complex subsystem interface	Reduced client complexity; decoupling

Short Questions

Q: What is the Observer pattern? Give a real-world example.

Observer defines one-to-many dependency. When Subject changes, all Observers are notified. Real-world example: A spreadsheet (Subject) and a bar chart (Observer). When data in spreadsheet changes, bar chart updates automatically.

Q: What is the Singleton pattern? Where is it used?

Singleton ensures only one instance of a class exists with a global access point. Used for: database connections, configuration managers, logging systems, thread pools.

Q: What is the Facade pattern?

Facade provides a simplified unified interface to a complex subsystem. It decouples clients from complex implementation classes. Example: a compiler's front-end API hiding scanner, parser, code generator complexity.

Q: In Observer pattern, who is notified when state changes?

All registered ConcreteObservers are notified when the Subject's state changes via the Notify() method which calls Update() on each observer.

Q: How does Facade reduce coupling?

Clients communicate with Facade instead of directly with multiple subsystem classes. This reduces dependencies between client code and internal subsystem classes.

MCQs

Q41. Observer pattern is also known as:

- (A) Command pattern
- ✓ (B) Publish-Subscribe pattern
- (C) Strategy pattern
- (D) Singleton pattern

Explanation: Observer = Publish-Subscribe mechanism.

Q42. Singleton ensures:

- (A) Many instances

(B) Zero instances

✓ (C) Exactly one instance with global access

(D) Unlimited instances

Explanation: Singleton = only one instance + global access point.

Q43. In Facade pattern, subsystem classes have knowledge of:

(A) The Facade

(B) All clients

✓ (C) Nothing about the Facade

(D) Other subsystems only

Explanation: Subsystem classes have NO knowledge of the Facade.

Q44. Which pattern would you use for a spreadsheet updating a bar chart automatically?

(A) Singleton

(B) Facade

✓ (C) Observer

(D) Factory

Explanation: Observer: subject (spreadsheet) notifies observers (charts).

Q45. In Singleton, the constructor is:

(A) Public

✓ (B) Private or Protected

(C) Abstract

(D) Virtual

Explanation: Protected constructor prevents external instantiation.

■ Memory Trick

Observer = One Subject, MANY Observers | Singleton = ONE and only ONE | Facade = ONE face for MANY classes

Observer is 1 → Many. Singleton is 1. Facade is many → 1. All about how "ONE" is used!

Lecture 28: Good Programming Practices – Maintainable Code

Definition: Self-Documenting Code

Code that explains itself without the need for comments or external documentation. The meaning of the code is evident just by reading it.

Fowler: "Any fool can write code that computers can understand, good programmers write code that humans can understand."

Three Basic Principles of Maintainability:

- Simplicity – easy to understand.
- Clarity – meaning is clear.
- Generality/Flexibility – easy to enhance.

1. Function Size

- Functions should NOT exceed 20 lines of code.
- In any case, should not exceed ONE PAGE.
- Reason: Reader should see entire function on one screen/page without scrolling.
- Longer functions = harder to understand = harder to maintain.

2. Identifier Names

Names must be meaningful. Progression of improvement:

Bad Code	Better Code	Best Code (Self-Documenting)
if (x==0)	if (AllocFlag == 0)	if (AllocFlag == NEW_NUMBER)

Using a named constant NEW_NUMBER instead of magic number 0 makes the code self-documenting.

3. Coding Style Guide – Naming Conventions

Hungarian Notation: Variable name includes type/scope information. **CamelCase:** compound words joined with each word capitalized. VU uses a mix of both.

Rule	Example
Type names: nouns, start with uppercase	Line, FilePrefix
Variable names: mixed case, start lowercase	line, filePrefix
Constants: ALL_UPPERCASE with underscores	MAX_ITERATIONS, COLOR_RED
Method names: verbs, start lowercase	getName(), computeTotalWidth()
Template types (C++): single uppercase letter	T, C, D
Global variables (C++): use :: operator	::mainWindow.open()
Private class variables: _ suffix	length_
Boolean methods: is/has/can/should prefix	isSet, hasLicense, canEvaluate
Accessor methods: get/set prefix	getName(), setName()
Compute methods: compute prefix	computeAverage()

■ Exam Tip

Negated boolean variable names MUST be avoided! `isNotError` is bad because `!isNotError` becomes double-negative and is confusing.

Short Questions

Q: What is self-documenting code?

Code that explains itself without requiring comments or external documentation. The meaning is evident from reading the code itself through good naming, structure, and formatting.

Q: What is Hungarian Notation?

A naming convention that includes type/scope information in the variable name itself. Allows the programmer to understand variable type just from its name. Example: `nCount` (`n`=integer), `szName` (`sz`=null-terminated string).

Q: What is CamelCase?

Writing compound words joined without spaces where each word is capitalized. Example: `FilePrefix`, `computeAverage`. Java uses this as its official convention.

Q: Why should functions be less than 20 lines?

20 lines approximately fits on one computer screen. The reader can see the entire function context without scrolling, making it easier to understand.

Q: What prefix should Boolean variables and methods use?

The prefix `is/has/can/should`. Examples: `isSet`, `hasLicense`, `canEvaluate`, `shouldAbort`. Avoid names like "flag" or "status" which don't describe what the boolean means.

MCQs

Q46. Which naming convention includes type information in the variable name?

- (A) CamelCase
- ✓ (B) Hungarian Notation
- (C) Underscore notation
- (D) Abbreviated naming

Explanation: Hungarian Notation includes type info in variable name.

Q47. Constants in C++ and Java should be named:

- (A) camelCase
- (B) PascalCase
- ✓ (C) ALL_UPPERCASE_WITH_UNDERSCORES
- (D) lowercase

Explanation: Constants: `MAX_ITERATIONS`, `COLOR_RED`.

Q48. The ideal maximum length of a function is:

- (A) 5 lines
- (B) 10 lines
- ✓ (C) 20 lines
- (D) 100 lines

Explanation: 20 lines = one screen. Should not exceed one page.

Q49. Private class variables in C++ should have:

- (A) No special marker
- (B) `_` prefix
- ✓ (C) `_` suffix
- (D) `ALL_CAPS`

Explanation: Private vars: `length_` (underscore suffix).

Q50. Which of the following is BEST for a self-documenting boolean check?

- (A) `if(x==0)`
- (B) `if(flag)`
- ✓ (C) `if(isNewNumber)`
- (D) `if(n)`

Explanation: `isNewNumber` clearly describes the intent.

■ Memory Trick

TYPES Start BIG, variables start small, CONSTANTS SHOUT

Types: `LineData` (Big start). variables: `lineData` (small start). CONSTANTS: `MAX_SIZE` (SHOUT). Easy!

Lecture 29: File Handling, Classes & Variable Guidelines

1. File Handling (C++ and Java)

- C++ header files: extension .h (MyClass.h)
- C++ source files: .c++, .C, .cc, or .cpp (MyClass.c++)
- Classes should be declared in individual header files matching the class name.
- All definitions (implementations) should reside in SOURCE files, not header files.
- Exception: inline functions must be defined in header files.
- Avoid special characters like TAB and page breaks (cause problems in multi-platform environments).
- Split lines must make incompleteness obvious; break after comma or operator.

2. Include Files

Header files must use include guards to prevent multiple inclusion:

```
#ifndef MOD_FILENAME_H #define MOD_FILENAME_H ... #endif
```

This prevents compilation errors from double-inclusion. Place at the top of the file to abort parsing immediately.

3. Classes and Interfaces – Organization Order

- 1. Class/Interface documentation (Javadoc/Doxygen comments).
- 2. class or interface statement.
- 3. Class (static) variables: public → protected → package → private.
- 4. Instance variables: public → protected → package → private.
- 5. Constructors.
- 6. Methods (no specific order required).

4. Types and Type Conversions

- Type conversions must ALWAYS be done EXPLICITLY (never rely on implicit conversion).
- Types local to one file can be declared inside that file.
- Parts of a class sorted: public first, then protected, then private ("most public first").

```
floatValue = (float) intValue; // CORRECT - explicit floatValue = intValue; // WRONG - implicit conversion
```

5. Variables

- Initialize variables where declared; declare in smallest scope possible.
- Variables must NEVER have dual meaning (one variable = one concept).
- Class variables should NEVER be declared public (violates encapsulation).
- Keep variables alive for as short a time as possible.
- Do NOT use global variables.
- Related variables of same type can share a declaration: float x, y, z;
- Implicit test for 0 should only be used for boolean variables and pointers, NOT for integers or floats.

```
if (nLines != 0) // CORRECT for int if (nLines) // WRONG - unclear type intent
```

6. Loop Structures

- Only loop control statements in the for() construction (no initialization of unrelated vars).
- Loop variables should be initialized IMMEDIATELY BEFORE the loop.
- Avoid do-while loops (harder to read; any do-while can be written as while or for).
- Avoid break and continue in loops (reduce readability).

- Use `for(;;)` for empty/infinite loops, NOT `while(true)`.

7. Conditionals

- Avoid complex conditional expressions; use temporary boolean variables.
- Put the NOMINAL case in if-part; EXCEPTION in else-part.
- Put the conditional on a SEPARATE LINE from the body.
- Avoid executable statements inside conditionals.

Short Questions

Q: What is an include guard and why is it needed?

An include guard is an `#ifndef/#define/#endif` construction in C++ header files. It prevents the header from being included multiple times in the same compilation unit, avoiding compilation errors.

Q: Why should class variables never be public?

Public variables violate encapsulation and information hiding. Instead, use private variables with public get/set accessor methods. Exception: if the class is purely a data structure (equivalent to C struct).

Q: Why should do-while loops be avoided?

Do-while loops have the condition at the end, making them harder to read than while/for loops where the condition is at the top. Any do-while can be rewritten as a while or for loop.

Q: What is the organization order of parts within a class?

1) Documentation, 2) class/interface statement, 3) static variables (public→private), 4) instance variables (public→private), 5) Constructors, 6) Methods.

Q: Why should variables be declared in the smallest scope possible?

Smaller scope means fewer places where the variable can be modified (fewer side effects). It also makes the code easier to understand and debug.

MCQs

Q51. C++ header files should have extension:

- (A) .cpp
- ✓ (B) .h
- (C) .c
- (D) .obj

Explanation: Header files use .h extension.

Q52. Include guards prevent:

- (A) Syntax errors
- ✓ (B) Multiple inclusion of header files
- (C) Runtime errors
- (D) Memory leaks

Explanation: Include guards prevent multiple-inclusion compilation errors.

Q53. Class variables should never be declared:

- (A) Protected
- (B) Private

✓ (C) Public

(D) Static

Explanation: Public class variables violate encapsulation.

Q54. The do-while loop should be avoided because:

(A) It is slower

✓ (B) Condition is at end, harder to read

(C) It cannot be nested

(D) It has no condition

Explanation: Condition at end = harder to read; prefer while/for.

Q55. Implicit test for 0 should only be used for:

(A) Integer variables

(B) Float variables

✓ (C) Boolean variables and pointers

(D) All variables

Explanation: Only for booleans and pointers, NOT int or float.

■ Memory Trick

PUBLIC first in class = OPEN DOOR policy

Arrange class members: Public (open to all), then Protected (family only), then Private (personal). Like floors of a building.

Lecture 30: Comments, Layout & Natural Form of Expressions

1. The Problem with Comments

"The problem with comments is that they LIE. Comments are not syntax checked; nothing forces them to be accurate."
– Fowler

- Comments become inaccurate as code changes during schedule crunches.
- Comments should NOT be used as deodorants (to cover up bad code).
- Tricky code should be REWRITTEN, not commented.
- Use comments minimally by making code self-documenting.

Comment Guidelines

- Write all comments in English (international environment).
- Use // for ALL comments, including multi-line (enables /* */ for debugging).
- Comments should be indented relative to their position in the code.

```
while (true) { // Do something <- CORRECT: indented something(); }
```

2. Layout Guidelines

- Basic indentation: 2 spaces (not 1 – too small; not >4 – causes line splits).
- 2 is chosen to reduce chance of line splits; readable on most editors.

3. Natural Form for Expressions

Write expressions as you would say them aloud. Conditional expressions with negation are ALWAYS difficult to understand:

```
// HARD to understand: if (!(block < activeBlock) || !(blockId >= unblocks)) // NATURAL form
(equivalent): if ((block >= activeBlock) || (blockId < unblocks))
```

4. Use Parentheses to Remove Ambiguity

Always use parentheses when mixing operators. Operator precedence is often misremembered:

```
// WRONG: == has higher precedence than & if (x & MASK == BITS) // parsed as: x & (MASK==BITS) //
CORRECT: parentheses clarify if ((x & MASK) == BITS)

// Leap year WITHOUT parentheses (unclear): leapYear = year % 4 == 0 && year % 100 != 0 || year %
400 == 0; // WITH parentheses (clear definition): leapYear = ((year%4==0) && (year%100!=0)) ||
(year%400==0);
```

5. Break Up Complex Expressions

An expression is complex if it uses MANY operators. Break it into multiple statements:

```
// Complex (hard to understand): x += (*xp=(2*k < (n-m) ? c[k+1] : d[k--])); // Simple equivalent:
if (2*k < n-m) *xp = c[k+1]; else *xp = d[k--]; *x = *x + *xp;
```

6. Shortcuts and Cryptic Code

Programmers sometimes write very concise (cryptic) code that is hard to maintain:

```
// Cryptic (what does this do?): subkey = subkey >> (bitoff - (bitoff >> 3) << 3)); // Clear
equivalent: subkey = subkey >> (bitoff & 0x7); // mask with octal 7 // Cryptic bit shift: a = a >>
2; // Clear intent: a = a / 4; // divide by 4
```

■ Important Note

NEVER write cryptic code for cleverness. Future maintainers (including yourself) will struggle. Always code for readability over cleverness.

Short Questions

Q: Why do comments "lie" according to Fowler?

Comments are not syntax-checked; nothing forces them to remain accurate. As code changes over time, comments are often not updated, so they become misleading or incorrect.

Q: Why should `//` be used instead of `/* */` for comments?

Because C++ and Java don't support multi-level commenting with `/* */`. Using `//` means you can always use `/* */` to comment out entire sections for debugging without conflicts.

Q: What is the "natural form" principle for expressions?

Write expressions as you would say them aloud. Avoid double negatives and confusing operator orders. Transform negative conditions to their natural positive equivalent for readability.

Q: Why are parentheses important in complex expressions?

Operator precedence is often misremembered. Parentheses eliminate ambiguity and clearly show grouping, preventing logical errors. Example: `(x & MASK) == BITS` vs `x & MASK == BITS` have different meanings.

Q: What basic indentation should be used?

Basic indentation of 2 spaces. 1 space is too small to show structure. More than 4 causes line splits in deeply nested code.

MCQs

Q56. The main problem with comments is that they:

- (A) Are hard to type
- ✓ (B) Are not syntax-checked and can become inaccurate
- (C) Take too much space
- (D) Slow down compilation

Explanation: Comments lie because code changes but comments are not updated.

Q57. Recommended basic indentation level is:

- (A) 1 space
- ✓ (B) 2 spaces
- (C) 4 spaces
- (D) 8 spaces

Explanation: 2 spaces: balance between visibility and line length.

Q58. In C++, if `(x & MASK == BITS)` has a bug because:

- (A) `&` has higher precedence than `==`
- ✓ (B) `==` has higher precedence than `&`
- (C) Both have same precedence
- (D) Neither is valid

Explanation: `==` has higher precedence than `&`, so it evaluates `MASK==BITS` first.

Q59. Which is the BEST approach to cryptic code?

✓ (A) Add comments to explain it

- (B) Keep it to save lines
- (C) Rewrite it clearly
- (D) Leave it as is

Explanation: Rewrite cryptic code clearly; if needed add brief comments.

Q60. According to Fowler, tricky code should be:

- (A) Commented thoroughly
- (B) Deleted

✓ (C) Rewritten for clarity

- (D) Left to the reader to understand

Explanation: Tricky code should be rewritten, not commented.

■ Memory Trick**Comments LIE, Code NEVER LIES**

Make code tell its own story. Write names and structures so clearly that comments become unnecessary. "Code is documentation."

Lecture 31: Switch Statements, Magic Numbers & Use of Zero

1. Switch Statement Guidelines

In switch statements, cases should ALWAYS end with a break. Fall-through (missing break) causes cryptic behavior:

```
// BAD: fall-through causes confusion switch(c) { case '-': sign = -1; case '+': c = getchar(); // '-' also executes this! case '.': break; } // GOOD: explicit breaks switch(c) { case '-': sign = -1; c = getchar(); break; case '+': c = getchar(); break; case '.': break; default: if (!isdigit(c)) return 0; break; }
```

■ Exam Tip

Even better: replace switch with if-else when the logic is simple. Explicit is always better than relying on fall-through.

2. Magic Numbers

Definition: Magic Number

A numeric literal (constant) that appears in code without any explanation of its meaning. Magic numbers make code hard to understand and modify.

Example of magic numbers (BAD):

```
fac = lim / 20; // What is 20? for (i=0, col=0; i<27; i++, j++) { // What is 27? col += 3; // What is 3?
```

Same code with NAMED CONSTANTS (GOOD):

```
enum { MINROW=1, MINCOL=1, MAXROW=24, MAXCOL=80, NLET=26, HEIGHT=MAXROW-4 }; fac = (lim+HEIGHT-1) / HEIGHT;
```

Named constants make the code self-documenting. The reader immediately understands intent.

3. Use (or Abuse) of Zero

In old-style C/C++, 0 is used for everything. This is confusing:

```
// BAD: 0 used for everything flag = 0; // flag is boolean str = 0; // str is string pointer name[i] = 0; // name is char array x = 0; // x is float i = 0; // i is integer // GOOD: use appropriate symbols flag = false; str = NULL; name[i] = '\0'; x = 0.0; i = 0;
```

■ Exam Tip

Using type-appropriate zero symbols (false, NULL, '\0', 0.0) makes the code self-documenting and avoids confusion about the variable type.

Short Questions

Q: What is a magic number? Why should it be avoided?

A magic number is a numeric literal (e.g., 20, 27, 3) appearing in code without explanation. It should be replaced with a named constant because the constant name explains the purpose, making code readable and easier to modify.

Q: What is the problem with switch statement fall-through?

When a case lacks a break, execution "falls through" to the next case. This causes unintended execution of subsequent cases, leading to cryptic, hard-to-debug behavior.

Q: Why is using 0 for all zero-like values bad practice?

In C/C++, 0 is used for boolean false, NULL pointer, null character, integer 0, and float 0.0. Using 0 for all these makes the code ambiguous. Using false, NULL, '\0', 0.0 makes intent clear.

Q: What is the better alternative to a complex switch statement?

An if-else chain. It is more explicit, easier to read, and does not have the fall-through problem.

Q: Name three named constants from the "fac" code example.

MINROW (1), MAXROW (24), NLET (26), HEIGHT (MAXROW-4), MAXCOL (80). These replace magic numbers like 20, 27, 3 in the original code.

MCQs**Q61. What is a magic number in programming?**

- (A) A very large number
- ✓ (B) A numeric literal without explanation in code
- (C) A random number
- (D) A number used in cryptography

Explanation: Magic number = unexplained numeric literal in code.

Q62. Switch case fall-through occurs when:

- (A) Case is empty
- ✓ (B) break statement is missing
- (C) default is absent
- (D) The switch value is 0

Explanation: Missing break causes fall-through to next case.

Q63. To represent a null character in a char array, use:

- (A) '0'
- (B) NULL
- ✓ (C) '\0'
- (D) false

Explanation: Null char in char array = '\0', not 0 or NULL.

Q64. Named constants help primarily with:

- (A) Performance
- (B) Memory usage
- ✓ (C) Code readability and maintainability
- (D) Compilation speed

Explanation: Named constants explain meaning, improving readability.

Q65. A boolean flag variable set to 0 should instead be set to:

- (A) NULL
- (B) \0
- (C) 0.0
- ✓ (D) false

Explanation: Boolean variables should use false (not 0).

■ Memory Trick

NO Magic Numbers = NO Magic Problems

Every magic number you leave is a mystery for the next developer. Replace 42 with MAX_STUDENTS. Future you will be grateful!

Master Revision – Lectures 24–28

Concept	Key Point
Repository Model	Central DB shared by all; efficient for large data sharing
Client-Server	Distributed; clients call services from servers
Thin Client	All processing at server; heavy server load
Fat Client	Processing at client; installation overhead
Zero Install	No client installation; web-based; server-side updates
N-Tier	Multiple server layers for scalability
Pipes & Filters	Independent filters connected by pipes; data flow architecture
Layered Architecture	Layers isolate complexity; great for portability
Design Pattern	Reusable solution to recurring design problem
GoF Categories	Creational, Structural, Behavioral
Observer	One-to-many notification on state change
Singleton	One instance only, global access point
Facade	Simplified interface to complex subsystem
Self-Documenting Code	Code explains itself via naming, structure
Magic Numbers	Unexplained numeric literals; replace with named constants

Lecture 32: Clarity Through Modularity, Short-Circuit Operators & Side Effects

1. Clarity Through Modularity

Break large functions into smaller, focused sub-functions. Selection Sort Example:

After breaking selectionSort into swap() and minimum() sub-functions:

- selectionSort becomes shorter and easier to read.
- swap() and minimum() are now reusable.
- Each function has a single, clear responsibility.
- Modularity is valuable even when sub-functions are not reused.

QuickSort example: The complex partitioning logic extracted into partition() makes quickSort trivially readable:

```
void quickSort(int a[], int left, int right) { int p; if (left < right) { p = partition(a, left, right); quickSort(a, left, p-1); quickSort(a, p+1, right); } }
```

■ Exam Tip

Break functions into smaller pieces even if those pieces are NOT reused. Modularity for readability is as important as modularity for reusability.

2. Short-Circuit || and &&

Definition: Short-Circuit Evaluation

In C/C++, logical expressions (|| and &&) are evaluated left to right and evaluation STOPS as soon as the final truth value can be determined.

Example of WRONG guard placement:

```
// WRONG: ptr checked AFTER dereferencing while (ptr->data < myData && ptr != NULL) { ... } //  
CORRECT: guard (ptr!=NULL) checked FIRST while (ptr != NULL && ptr->data < myData) { ... }
```

If ptr is NULL and guard is on the right, ptr->data is evaluated FIRST, causing a crash. Guard MUST be on the left.

3. Operand Evaluation Order and Side Effects

Definition: Side Effect

When a function, besides returning a value, also modifies a parameter or external variable. Side effects + unspecified evaluation order = major bugs.

Example: `c = f1(a) + f2(a);` where f1 and f2 both modify their argument. Results differ based on which function evaluates first. This is UNDEFINED BEHAVIOR in C/C++.

Guidelines to avoid side effect bugs:

- Never use "," (comma operator) except for declarations.
- If initializing a variable at declaration, do not declare another in the same statement.
- Never use multiple assignments in the same statement.
- Be very careful with functions that change parameter values.
- Try to avoid functions that change parameters AND return a value simultaneously.

■ Important Note

`int i, j = 0;` — Many assume `i` is also 0, but only `j` is initialized. `i` has garbage value! This is a common C++ trap.

Short Questions

Q: Why is modularity important even when sub-functions are not reused?

Modularity reduces function size, making each function fit on one screen. This improves readability and maintainability. When logic is extracted into named sub-functions, the intent of the parent function becomes immediately clear.

Q: What is short-circuit evaluation?

In C/C++, `||` and `&&` stop evaluating as soon as the result is determined. For `||`, if the left is true, right is skipped. For `&&`, if the left is false, right is skipped. Used for guard conditions.

Q: Why must the NULL pointer guard appear FIRST in a while condition?

Because of short-circuit evaluation: if `ptr!=NULL` is on the LEFT and `ptr` IS NULL, the right side (`ptr->data`) is never evaluated, preventing a crash. If guard is on the right, `ptr->data` is evaluated first, causing undefined behavior.

Q: What is a side effect in programming?

A side effect occurs when a function, besides returning its explicit value, also modifies a parameter or external variable. Side effects combined with unspecified evaluation order cause unpredictable bugs.

Q: What happens in `int i, j = 0`?

Only `j` is initialized to 0. `i` is uninitialized (contains garbage value). Many programmers incorrectly assume both are initialized to 0. This is a dangerous misunderstanding.

MCQs

Q66. Short-circuit evaluation means:

- (A) All operands are always evaluated
- ✓ (B) Evaluation stops when result can be determined
- (C) Only first operand is evaluated
- (D) No operands are evaluated

Explanation: Short-circuit: evaluation stops as soon as result is known.

Q67. In `while (ptr->data < myData && ptr != NULL)`, what is wrong?

- (A) Missing semicolon
- ✓ (B) `ptr` is dereferenced before NULL check
- (C) `myData` is undefined
- (D) Wrong operator

Explanation: NULL check must come FIRST to prevent crash on null pointer.

Q68. A side effect of a function is:

- (A) Its return value
- ✓ (B) A modification of parameters or external variables
- (C) A runtime error
- (D) A compilation warning

Explanation: Side effect = modifying something beyond the return value.

Q69. In `int i, j = 0`, which variable is initialized to 0?

- (A) Both `i` and `j`

(B) Only i

✓ (C) Only j

(D) Neither

Explanation: Only j = 0. i has garbage value.

Q70. Breaking a large function into smaller ones primarily improves:

(A) Runtime speed

(B) Memory usage

✓ (C) Readability and maintainability

(D) Compilation speed

Explanation: Modularity = readability + maintainability.

■ Memory Trick

GUARD comes FIRST in && conditions

Like a security guard: the guard checks FIRST before allowing access. `ptr != NULL` guards `ptr->data`. Guard first, then dereference.

Lecture 33: Common Coding Mistakes & Performance

1. Common Coding Mistakes from Side Effects

Most dangerous coding mistakes due to side effects:

```
1. array[i++] = i; // i might be 3 or 4 depending on compiler
2. array[i++] = array[i++] = x; // multiple side effects: undefined
3. int i, j = 0; // i is NOT initialized (garbage)
4. a = b, c = 0; // only c gets 0; a and b have garbage values
```

2. Guidelines to Avoid Side Effect Bugs

- Never use "," except for variable declarations.
- Never initialize multiple unrelated variables in one statement.
- Never use multiple assignments in one statement.
- Avoid functions with side effects (change parameters AND return a value).

3. Performance – The 80/20 Rule

Definition: 80/20 Rule (Pareto Principle)

In most programs, 80% of execution time is spent in just 20% of the code. Therefore, only optimize the bottleneck (hot spots), not the entire codebase.

Profiling: Measure where time is actually being spent. Don't guess. Profile first, then optimize.

Example – issipam function profiling (10,000 calls):

Function	Cycles	Finding
strchr	990,000	Most time spent here → BOTTLENECK
strncmp	80,000	Second most – also called from strstr
strstr	(calls strchr and strncmp)	Root cause of bottleneck

Solution: Rewrite strstr to eliminate strchr and strncmp. Replace with memcmp. Result: strlen calls dropped from 2 million to 652.

Key lessons:

- Most time is spent in loops – concentrate on loops.
- The trick: find hot spots first, then "cool them" (optimize).
- Example: loop clearing MAX_FIELD=200 fields when only 2-3 needed clearing → change to actual max needed.)
- Performance and maintainability are often at odds – profile before optimizing.

■ Exam Tip

This is analogous to database normalization vs. de-normalization for performance: normalize first (clean code), then selectively optimize bottlenecks.

Short Questions

Q: What does the 80/20 rule say about performance optimization?

80% of execution time is in 20% of code. So only optimize the 20% hot spots (bottlenecks), not the entire codebase. Profile first to find those hot spots.

Q: What is profiling in software development?

Profiling measures how much time and resources each part of a program consumes during execution. It identifies bottlenecks so developers can focus optimization efforts on the right places.

Q: What is the danger of `array[i++] = i`?

The expression has undefined behavior in C/C++ because the side effect of `i++` and the read of `i` in the right-hand side have an unspecified order of evaluation. The result may be `array[3]=3` or `array[3]=4`.

Q: What was the main bottleneck in the isspam example?

`strchr` (990,000 cycles) was the biggest bottleneck, called from `strstr`. After profiling, `strstr` was rewritten using `memcmp`, which eliminated `strchr` and `strncmp` calls.

Q: Why should we profile before optimizing?

Without profiling, developers guess which code is slow. They often optimize the wrong parts. Profiling reveals the actual bottlenecks (20% of code using 80% of time), ensuring effort is spent where it matters.

MCQs

Q71. The 80/20 rule in performance means:

- (A) 80% of bugs are in 20% of code
- ✓ (B) 80% of time spent in 20% of code
- (C) 80% of code should be optimized
- (D) 20% of developers write 80% of code

Explanation: 80% execution time in 20% of code.

Q72. What is the FIRST step before optimizing performance?

- (A) Rewrite all code
- ✓ (B) Profile to find bottlenecks
- (C) Delete unused functions
- (D) Add more hardware

Explanation: Profile first, then optimize the actual bottlenecks.

Q73. `array[i++] = i` is dangerous because:

- (A) Array is full
- ✓ (B) Evaluation order of `i` is undefined
- (C) `i` is not initialized
- (D) Array index out of bounds

Explanation: Undefined behavior: `i++` side effect order vs read of `i`.

Q74. In the isspam example, which function was rewritten for performance?

- (A) `strchr`
- (B) `strcmp`
- ✓ (C) `strstr`
- (D) `memcpy`

Explanation: `strstr` was rewritten using `memcmp`.

Q75. Performance optimization should focus primarily on:

- (A) All code equally
- (B) Header files
- ✓ (C) Loops and bottlenecks (hot spots)
- (D) Comments

Explanation: Most time is in loops; optimize hot spots.

■ Memory Trick

Profile → Find 20% → Optimize 20%

Don't optimize blindly. Be a smart archer: profile first to find the target, THEN shoot (optimize).

Lecture 34: Portability – Writing Portable Code

Definition: Portability

The ability to transfer a software application from one environment (OS, hardware) to another with minimum effort. Achieved by writing code that does not assume specific hardware or platform characteristics.

Portability Guidelines

1. **Stick to the Standard:** Use ANSI/ISO standard C++. Use STL instead of vendor-specific extensions.
2. **Program in the Mainstream:** Always write function prototypes (double sqrt(double); NOT double sqrt();)

3. Data Type Sizes – Major Portability Issue:

```
int j = 20000, k = 30000; i = j + k; // works if int is 4 bytes // FAILS if int is 2 bytes (overflow)!
```

4. **Order of Evaluation:** As discussed in side effects, order varies between compilers. Avoid expressions that depend on a specific evaluation order.

5. Signedness of char:

```
char c; // unsigned (0-255) on some, signed (-128 to 127) on others c = getchar(); if (c == EOF) // FAILS if char is unsigned! // CORRECT: int c; // always use int for getchar() result c = getchar(); if (c == EOF) ...
```

6. **Arithmetic vs Logical Shift:** C/C++ does not specify if >> is arithmetic (copies sign bit) or logical (fills 0s). Java solved this with >>> for logical shift.

7. Byte Order (Endianness):

- Intel: lowest byte = most significant (little-endian)
- Motorola: highest byte = most significant (big-endian)
- Causes problems when exchanging binary data between heterogeneous machines.
- Solution: use TEXT for data exchange; avoid binary.

8. **Alignment:** C/C++ does not define alignment of items within structures. The address of int i in struct {char c; int i;} could be 2, 4, or 8 from the start. Do not use pointer arithmetic assuming specific alignment.

9. Bit Fields:

- Allow packing multiple data items into one machine word.
- Useful for flags, non-standard file formats.
- Problems: signedness, max bits, storage order all platform-dependent.
- Recommendation: Do NOT use bit fields for portable code.

■ Exam Tip

Portability rule: "Write once, run anywhere." Use standards, avoid platform-specific tricks, use text for data exchange.

Short Questions

Q: What is portability in software? Why is it important?

Portability is the ability to move software from one platform to another with minimum effort. Important because systems change hardware/OS over time; portable code avoids expensive rewrites.

Q: What is endianness and why does it cause portability problems?

Endianness is the byte order in which multi-byte data is stored. Intel (little-endian) stores lowest byte at lowest address; Motorola (big-endian) does the opposite. Binary data exchange between these architectures gives incorrect values.

Q: Why should getchar() result be stored in an int, not a char?

EOF is often -1. If char is unsigned (0-255), EOF (-1) can never equal char's value, so the EOF check fails. Using int ensures the full signed range including EOF can be represented.

Q: Why should bit fields be avoided in portable code?

Bit field behavior (signedness, max bits, storage order left-to-right or right-to-left) is platform-dependent. What works on one architecture may fail on another.

Q: What is the safe approach for data exchange between heterogeneous systems?

Use TEXT (ASCII/Unicode) for data exchange, not binary. Binary data is affected by endianness and alignment issues. Text is portable across all platforms.

MCQs**Q76. Portability issues with data type sizes occur because:**

(A) Variables have no types

✓ (B) Data type sizes vary across platforms

(C) All types are 4 bytes

(D) Types are dynamic

Explanation: int size varies: 2 bytes on 16-bit, 4 bytes on 32-bit systems.

Q77. The safest data type for storing getchar() result is:

(A) char

(B) unsigned char

✓ (C) int

(D) long

Explanation: int can hold all char values plus EOF (-1).

Q78. Intel architecture stores bytes in:

(A) Big-endian order

✓ (B) Little-endian order

(C) Mixed order

(D) No specific order

Explanation: Intel = little-endian.

Q79. Java introduced >>> operator to handle:

(A) Arithmetic shift

✓ (B) Logical shift

(C) Both shifts

(D) Neither

Explanation: >>> = logical (unsigned) right shift in Java.

Q80. Bit fields should NOT be used in portable code because:

(A) They waste memory

✓ (B) Their behavior is platform-dependent

(C) They cause logical errors

(D) They are deprecated

Explanation: Bit field layout and signedness are platform-dependent.

■ Memory Trick

PORTABILITY = Use Standards + Text Data + No Assumptions

When writing portable code: S-T-A-N (Stick to Standard, Text for exchange, Avoid platform Assumptions, No bit fields).

Lecture 35: Exception Handling – Safety Levels & Guarantees

Definition: Exception Handling

A mechanism that separates error-handling code from normal code. Provides consistent error handling. Key advantage: ability to handle asynchronous errors gracefully.

Exception Handling Mechanism

- **throw:** Raising/throwing an error condition.
- **catch:** Catching and handling the thrown error.
- **try block:** Surrounding error-sensitive code to attempt execution.

```
try { ... throw Exception(); // error raised ... } catch (Exception e) { // handle error here }
```

Key feature: errors can be thrown ACROSS function boundaries. This allows centralizing error handling (e.g., in main function).

Exceptions and Code Complexity

A simple-looking function with exceptions can have FAR MORE execution paths than apparent:

Example – EvaluateSalaryAndReturnName function: appears to have 3 non-exceptional paths but actually has 23 TOTAL paths (3 normal + 20 exceptional).

■ Exam Tip

ALWAYS be exception-aware. Know which operations might throw: copy constructors, operator==, operator<<, member function calls, string operations.

Levels of Exception Safety

Guarantee Level	What It Ensures
Basic Guarantee	No memory leaks; temporaries destroyed properly. Program remains in valid state.
Strong Guarantee	Basic guarantee PLUS full commit-or-rollback: either BOTH side effects complete or NEITHER does
No-throw Guarantee	Function will NEVER throw an exception.

Exception-safety requires either: No-throw guarantee, OR Basic + Strong guarantee.

Strong Guarantee Example

The original EvaluateSalaryAndReturnName does NOT satisfy strong guarantee because:

- A partial message might be emitted to cout before exception.
- A message might be emitted even though function later fails.

Attempt 3 uses auto_ptr to achieve strong guarantee:

```
auto_ptr EvaluateSalaryAndReturnName(Employee e) { auto_ptr result = new String(e.First()+"
"+e.Last()); if (e.Title()=="CEO" || e.Salary()>10000) { String message = (*result)+" is
overpaid\n"; cout << message; // first side-effect } return result; // non-throwing transfer of
ownership }
```

Key Exception Safety Concepts

Definition: Exception-Safe Function

A function is exception-safe if it might throw but has NO SIDE EFFECTS if it does throw, and properly cleans up temporaries.

Definition: Exception-Neutral Function

A function that propagates ALL exceptions to the caller (does not swallow exceptions).

■ Important Note

Providing strong exception-safety often requires a PERFORMANCE TRADE-OFF. Not all functions need strong exception safety – basic guarantee may be sufficient.

Short Questions

Q: What are the three levels of exception safety?

1) Basic: No leaks, temporaries destroyed. 2) Strong: Full commit-or-rollback (atomicity). 3) No-throw: Function never throws.

Q: What is the difference between exception-safe and exception-neutral?

Exception-safe: function might throw but has no side effects if it does. Exception-neutral: function propagates all exceptions to the caller without swallowing them.

Q: How many total execution paths does the EvaluateSalaryAndReturnName example have?

23 total paths: 3 non-exceptional paths (normal cases) and 20 exceptional paths (from various potential throw points).

Q: What is the strong guarantee?

Strong guarantee ensures that if a function fails due to an exception, the program state is UNCHANGED. It is full commit-or-rollback: either all side effects complete or none do.

Q: Why is auto_ptr used in attempt 3 of the salary example?

auto_ptr ensures that if an exception occurs, the allocated String is automatically destroyed (no memory leak). The return operation (transfer of ownership) is non-throwing, ensuring the strong guarantee.

MCQs

Q81. The "try" block is used to:

- (A) Declare variables
- ✓ (B) Surround error-sensitive code
- (C) Handle exceptions
- (D) Throw exceptions

Explanation: try { } surrounds code that might throw.

Q82. Which guarantee ensures full commit-or-rollback?

- (A) Basic Guarantee
- ✓ (B) Strong Guarantee
- (C) No-throw Guarantee
- (D) Weak Guarantee

Explanation: Strong guarantee = atomicity (all or nothing).

Q83. An exception-neutral function:

(A) Swallows exceptions

✓ (B) Propagates all exceptions to caller

(C) Handles all exceptions

(D) Never throws

Explanation: Exception-neutral = propagates exceptions up.

Q84. The EvaluateSalaryAndReturnName example has how many exceptional paths?

(A) 3

(B) 10

✓ (C) 20

(D) 23

Explanation: 20 exceptional + 3 normal = 23 total.

Q85. What is the BIGGEST advantage of exception handling?

(A) Faster execution

✓ (B) Separates error-handling from normal code; handles async errors

(C) Less memory usage

(D) Fewer function calls

Explanation: Error-handling code separated from normal code.

■ Memory Trick

Exception Safety = B-S-N (Basic, Strong, No-throw)

Think BSN like "Business Safety Network": Basic = no leaks, Strong = atomic, No-throw = bulletproof.

Master Revision – Lectures 29–33

Topic	Key Takeaway
Include Guards	#ifndef/#define/#endif prevents multiple inclusion
Class Organization	Static vars → Instance vars → Constructors → Methods
Natural Expressions	Write expressions as you'd say them; avoid double negatives
Parentheses	Always use for mixed operators; operator precedence traps
Magic Numbers	Replace all numeric literals with named constants
Use of Zero	false, NULL, '\0', 0.0 instead of 0 for different types
Modularity	Break functions into smaller; even if not reused
Short-Circuit	Guard condition must be FIRST in && chains
Side Effects	Avoid functions that both modify params AND return values
80/20 Rule	Profile first; 80% time in 20% code; optimize hot spots
Portability	Use ANSI standards; text for exchange; avoid bit fields
Exception Safety	Basic, Strong, No-throw guarantee levels

Lecture 36: Software Verification & Validation, Testing Concepts

1. Software Verification and Validation

Concept	Question It Answers	Meaning
Verification	"Have you built the product right?"	Does the product meet SYSTEM SPECIFICATIONS?
Validation	"Have you built the RIGHT product?"	Does the product meet USER EXPECTATIONS?

A software can be VERIFIED but not VALIDATED if user needs were not captured precisely during requirements engineering.

2. Defect

Definition: Defect

A variance from a desired product attribute. Any phenomenon that may cause customer dissatisfaction is a defect. Defects can be in specifications OR in the software itself.

"Death, taxes, and bugs are the only certainties in the life of a programmer." – Kernighan

3. Software Testing – Definition

Definition: Software Testing

The process of examining the software product against its requirements. It involves executing software on test data and comparing output against documented expected behavior.

Testing Objective

- "The goal of testing is to EXPOSE LATENT DEFECTS, not to prove their absence." – Popper 1965
- A tester tries to BREAK the system.
- Objective: show PRESENCE of defects, not ABSENCE.
- Testing CANNOT show absence of defects – it only increases CONFIDENCE.
- "A test is said to be SUCCESSFUL if it DISCOVERS AN ERROR." – Myers 1979

4. Limitations of Testing

Testing is limited because:

- The domain of possible inputs is too large to test exhaustively.
- There are too many possible paths through a program to test.
- Proving correctness requires formal proof, not just examples.

Example: `isStringsEqual` function passes ALL designed test cases but STILL fails for strings of same length ending with the same character (e.g., "cut" vs "rat" returns true incorrectly).

■ Important Note

Testing can NEVER prove a program is correct – it can only increase confidence or reveal bugs.

5. Test Cases and Test Data

Definition: Test Case

Input and output specification + statement of function under test + steps to perform + expected results.

Definition: Test Data

The actual input values devised to test the system.

Short Questions

Q: What is the difference between verification and validation?

Verification: "Have you built the product RIGHT?" – checking against specifications. Validation: "Have you built the RIGHT product?" – checking against user expectations. A system can be verified but not validated.

Q: What is a defect in software?

A defect is any variance from a desired product attribute that may cause customer dissatisfaction. Defects can exist in specifications, design, code, or documentation.

Q: What is the objective of software testing?

To expose latent defects (bugs) before software is put to use. Testing aims to SHOW PRESENCE of defects, not prove their absence. A successful test is one that FINDS a bug.

Q: Why can't testing prove software is correct?

Because exhaustive testing is impossible: too many inputs and too many execution paths. Each test only increases confidence. Formal mathematical proof is required to prove correctness, not testing.

Q: What is the difference between a test case and test data?

Test case: specification including input, expected output, and steps. Test data: the actual input values (data) used to execute the test case.

MCQs

Q86. Which question does VERIFICATION answer?

(A) "Built the right product?"

✓ (B) "Built the product right?"

(C) "Is testing complete?"

(D) "Is code written?"

Explanation: Verification = "built product right?" = meets spec.

Q87. A successful test is one that:

(A) Passes all test cases

✓ (B) Discovers an error

(C) Takes minimum time

(D) Has maximum coverage

Explanation: "Successful test discovers an error." – Myers.

Q88. Testing CAN show:

(A) Absence of all defects

✓ (B) Presence of defects

(C) Software is perfect

(D) No more bugs exist

Explanation: Testing can only show PRESENCE of bugs, not absence.

Q89. A defect is:

(A) Any feature request

✓ (B) Any variance from desired product attribute

(C) Any missing documentation

(D) A slow function

Explanation: Defect = variance from desired attribute causing dissatisfaction.

Q90. Why is exhaustive testing impossible?

(A) Too expensive

✓ (B) Too many inputs and paths to test

(C) No testers available

(D) Tools are limited

Explanation: Infinite inputs + infinite paths = exhaustive testing impossible.

■ **Memory Trick**

Testing FINDS bugs; Proof PREVENTS them

Testing is like a doctor diagnosing disease. Formal proof is like preventive medicine. Both are needed! Testing shows bugs are PRESENT; it cannot show they are ABSENT.

Lecture 37: Testing vs. Development, Testing Phases

1. Testing vs. Development

Development	Testing
Creative activity (building system)	Destructive activity (breaking system)
Objective: show program WORKS	Objective: show program DOES NOT WORK
Analyst/designer perspective	Independent perspective needed
Builds system from requirements	Validates system against requirements

Key insight: If the developer tests their own code, they carry the same misunderstandings into testing. Independent testing breaks this bias. "Another pair of eyes" catches what the author cannot.

Testing is a parallel activity: While developers code, testers analyze requirements, plan tests, and write test cases. Testing does not begin only after development.

2. Usefulness of Testing

- Testing prevents costly defect discovery by the CLIENT.
- Defects found late cost orders of magnitude more to fix than those found early.
- Without testing: at best, client demands free fixes; at worst, lawsuits.
- "A good tester has a knack of smelling errors – just like auditors."

3. Testing Phases

Phase	What Is Tested
Unit Testing	Individual components (functions, classes) in isolation.
Module Testing	A collection of dependent components (a module). Tested independently.
Sub-system Testing	Collection of modules; discovers interfacing problems between modules.
System Testing	Complete integrated system tested as a whole.
Acceptance Testing	Validation against user expectations; usually at client site.
Alpha Testing	Acceptance testing for customized projects; in-house testing for products.
Beta Testing	Field testing with potential customers before general release.

4. Black Box vs. White Box Testing

Definition: Black Box Testing

Testing component/system without knowledge of internal implementation. Tests REQUIRED BEHAVIOR only (input → expected output). The internals are hidden to the tester.

Definition: White Box (Structural) Testing

Testing by examining the INTERNAL STRUCTURE: code design, structure, and documentation. Test cases designed to cover internal paths, branches, and statements.

Aspect	Black Box	White Box
Knowledge required	Only specification/interface	Internal code and design
Tests	Required behavior	Internal paths, branches, statements

Also called	Functional testing	Structural testing
Perspective	User's view	Developer's view

Short Questions

Q: Why should developers NOT test their own code?

Developers carry their own misunderstandings and assumptions into testing. They see what they intended, not what is there. An independent tester brings a fresh, unbiased perspective that can discover what the developer cannot see.

Q: List the 7 testing phases in order.

1) Unit Testing, 2) Module Testing, 3) Sub-system Testing, 4) System Testing, 5) Acceptance Testing, 6) Alpha Testing, 7) Beta Testing.

Q: What is black box testing?

Testing where the system is treated as a black box. The tester gives inputs through the provided interface and checks outputs against expected results. Internal implementation is hidden.

Q: What is white box testing?

Testing based on the internal structure of the code. Test cases are designed to cover code statements, branches, and paths. Also called structural testing.

Q: When does testing activity begin in the software lifecycle?

Testing begins in PARALLEL with development. While developers analyze, design, and code, testers analyze requirements, create test plans, and write test cases. Tests are executed after coding.

MCQs

Q91. Which testing phase discovers interfacing problems between modules?

- (A) Unit Testing
- (B) Module Testing
- ✓ (C) Sub-system Testing
- (D) System Testing

Explanation: Sub-system Testing finds module interfacing problems.

Q92. Beta testing involves:

- (A) In-house testing only
- ✓ (B) Testing by potential customers before release
- (C) Testing individual modules
- (D) Testing the database only

Explanation: Beta testing = field testing with real customers before release.

Q93. Black box testing is also called:

- (A) Structural testing
- (B) White box testing
- ✓ (C) Functional testing
- (D) Unit testing

Explanation: Black box = functional testing (tests required behavior).

Q94. The development team's objective is to show:

(A) Program does not work

✓ (B) Program works

(C) All bugs are found

(D) Specifications are wrong

Explanation: Developers try to show program works.

Q95. Alpha testing is:

(A) In-field testing by real users

✓ (B) In-house testing for products (before beta)

(C) Unit testing by developer

(D) Database testing

Explanation: Alpha = in-house testing for products.

■ Memory Trick

Testing Phases: U-M-S-S-A-A-B

Unit → Module → Sub-system → System → Acceptance → Alpha → Beta. Remember: "Uncle Makes Steaks, Sister Always Asks Boldly."

Lecture 38: Equivalence Partitioning & Flow Graphs

1. Equivalence Partitioning

Definition: Equivalence Class

A set of test inputs that are considered equivalent in that: if one discovers a defect, the others probably will too; if one does not, the others probably won't either.

Equivalence partitioning helps select a representative set of test cases efficiently.

Example – String Matching Function:

Partition	Test Cases
Equal strings – all lowercase	"cat" vs "cat"
Equal strings – all uppercase	"CAT" vs "CAT"
Equal strings – mixed case	"Cat" vs "Cat"
Equal strings – numeric	"123" vs "123"
Equal strings – with blanks	" " vs " "
Equal strings – NULL (empty)	"" vs ""
Unequal – different length	"cat" vs "mouse"
Unequal – same length	"cat" vs "dog"
Unequal – case difference	"Cat" vs "caT"
Unequal – one NULL	"cat" vs "" OR "" vs "cat"

Equivalence partitioning guidelines:

- Organize equivalence classes in some order (hierarchical or grouped by similarity).
- Determine boundary conditions (first/last element, empty, max size).
- Don't forget INVALID inputs (wrong type, out-of-range values).
- Finding equivalence classes is SUBJECTIVE – two people may produce different sets.

2. Basic Coding Structures (for Flow Graphs)

Definition: Flow Graph

A directed graph representing the control flow of a program. Nodes = statements or groups of statements. Edges = control flow (sequence, branches, loops).

Structure	Flow Graph Description
Sequence	Multiple sequential instructions lumped into one node.
If statement	3 nodes: if condition node → true branch → false branch → merge node.
Case statement	One node (switch) → multiple branches → merge.
While loop	Loop guard node → loop body → back edge to guard; exit edge.

Bubble Sort Flow Graph has 6 nodes:

- Node 1: while (!sorted) - outer loop guard
- Node 2: for(i=0; i<N-1; i++) - inner loop guard

- Node 3: swap(a[i], a[i+1]) - swap instruction
- Node 4: - after swap check
- Node 5: - end of for loop
- Node 6: - end of while loop

Short Questions

Q: What is equivalence partitioning?

A testing technique that divides the input domain into classes (partitions) of equivalent test cases. Test cases in the same class are expected to expose the same bugs. Testing one from each class is sufficient.

Q: What is a flow graph?

A directed graph representing the control flow of a program. Nodes represent statements or groups; edges represent flow of control. Used in structural (white-box) testing to analyze program paths.

Q: What are boundary conditions in equivalence partitioning?

Boundary conditions are edge cases at the limits of equivalence classes. Examples: empty list (0 elements), one element, maximum size, first element, last element, element just before/after boundaries.

Q: Name four basic coding structures represented in flow graphs.

1) Sequence (sequential statements). 2) If statement (two branches). 3) Case statement (multiple branches). 4) While loop (iterative structure with back edge).

Q: Why is finding equivalence classes subjective?

Different testers may analyze the same program differently and identify different partitions. There is no single "correct" set of equivalence classes; the quality depends on the tester's understanding of the system.

MCQs

Q96. In equivalence partitioning, tests in the same class are expected to:

- (A) Find different bugs
- ✓ (B) Find the same bugs or none
- (C) Always find a bug
- (D) Always pass

Explanation: Same equivalence class = same defect discovery behavior.

Q97. Which equivalence class tests for boundary condition of empty string?

- (A) Equal strings – numeric
- ✓ (B) Equal strings – NULL ("")
- (C) Unequal strings – same length
- (D) Any string

Explanation: Empty string "" is a special/boundary case.

Q98. In a flow graph, nodes represent:

- (A) Developers
- ✓ (B) Statements or groups of statements
- (C) Test cases
- (D) Variables

Explanation: Nodes = statements/groups; edges = control flow.

Q99. In a While loop flow graph, the back edge goes:

(A) To the next statement

✓ (B) Back to the loop guard

(C) To the exit

(D) To the function start

Explanation: Back edge returns control to the loop guard condition.

Q100. Finding equivalence classes is:

(A) An automated process

(B) Purely mathematical

✓ (C) Subjective (depends on tester)

(D) Always the same for all testers

Explanation: Subjective – different testers produce different partitions.

■ Memory Trick

Equivalence = "Same destiny" group

Tests in one equivalence class have the same destiny: they either ALL find the bug or NONE do. Test ONE from each group.

Lecture 39: White Box Testing & Cyclomatic Complexity

1. White Box Testing Coverage Schemes

Coverage Type	What Is Covered	Strength
Statement Coverage	Every statement on at least one path is executed.	Weakest – may miss uncovered branches
Branch Coverage	Every decision branch (true/false) is tested at least once	Stronger – covers all branches
Path Coverage	Every possible path from start to end is tested.	Strongest – exhaustive; often infeasible

Example with if statement:

```
if (a == b) c = d; // node 2 a++; // node 3
```

Statement Coverage: a=1, b=1 (only 1 test case) Branch Coverage: a=1,b=1 AND a=1,b=2 (2 test cases) Path Coverage: same as Branch Coverage here

2. Paths in Programs with Loops

For a loop with an if statement inside (N iterations, 2 branches):

- N=0: 1 path (skip loop)
- N=1: 2 paths
- N=2: 4 paths
- N=k: 2^k paths
- N=20: over 1 MILLION paths!

■ Important Note

Exhaustive path testing of programs with loops is IMPOSSIBLE due to the exponential explosion of paths. Use Cyclomatic Complexity instead.

3. Cyclomatic Complexity

Definition: Cyclomatic Complexity $V(G)$

A quantitative measure of the logical complexity of a program. Defines the number of INDEPENDENT PATHS in the basis set. Gives an UPPER BOUND on the number of tests needed to cover all statements and branches at least once.

Formula

$V(G) = E - N + 2$ Where: E = number of EDGES in flow graph N = number of NODES in flow graph

Bubble Sort Example:

- Edges E = 8
- Nodes N = 6
- $V(G) = 8 - 6 + 2 = 4$
- So 4 test cases are needed to cover all paths.

The 4 independent paths:

Path 1: 1-6 (sorted, skip everything) Path 2: 1-2-3-4-5-1-6 (loop once, swap) Path 3: 1-2-4-5-1-6 (loop once, no swap) Path 4: 1-2-4-2-3-4-5-6-1 (inner loop multiple)

4. Infeasible Paths

Definition: Infeasible Path

A path through a program that is NEVER traversed for any input data. It exists in the flow graph but cannot be executed.

Example: if (a==b) c=c-1; if (a!=b) c=c+1; — the path that goes through BOTH if-bodies is infeasible (a cannot equal b AND not equal b simultaneously).

Best practice: Minimize infeasible paths by using if-ELSE (not two separate if statements). This reduces the total number of test cases needed.

Short Questions

Q: What is statement coverage?

Statement coverage ensures every statement in the program is executed by at least one test case. It is the weakest form of coverage as it may not test all branches.

Q: What is cyclomatic complexity? Give the formula.

Cyclomatic complexity $V(G)$ measures logical program complexity and gives the upper bound on independent paths. Formula: $V(G) = E - N + 2$, where E = edges and N = nodes in the flow graph.

Q: For a loop with N=20 iterations containing one if statement, how many paths exist?

2^{20} = over 1 million paths. This shows why exhaustive path testing is infeasible for programs with loops.

Q: What is an infeasible path?

An infeasible path is a path through the program's flow graph that can never be executed for any input. It is mathematically possible in the graph but logically impossible in execution.

Q: How can infeasible paths be minimized?

By using if-ELSE instead of two separate if statements. With if-else, exactly one branch executes per decision point, eliminating infeasible paths.

MCQs

Q101. Cyclomatic complexity is calculated as:

- (A) $N - E + 2$
- ✓ (B) $E - N + 2$
- (C) $E + N - 2$
- (D) $E * N$

Explanation: $V(G) = E - N + 2$.

Q102. For bubble sort with E=8, N=6, cyclomatic complexity is:

- (A) 2
- ✓ (B) 4
- (C) 6
- (D) 8

Explanation: $V(G) = 8 - 6 + 2 = 4$.

Q103. Branch coverage tests:

- (A) Only the happy path

(B) All statements once

✓ (C) Every true/false branch

(D) Every possible path

Explanation: Branch coverage = test all decision branches.

Q104. An infeasible path is:

(A) A path that always executes

✓ (B) A path impossible to execute for any input

(C) A path not in flow graph

(D) A path that causes crashes

Explanation: Infeasible path exists in graph but cannot be executed.

Q105. For N=20 loop iterations with 2 branches per iteration, number of paths is:

(A) 20

(B) 40

✓ (C) 2^{20} (>1 million)

(D) 200

Explanation: 2^N paths for N iterations with binary branch.

■ Memory Trick

$V(G) = \text{Edges} - \text{Nodes} + 2$

Remember: "Every Node subtracts 2 from Edges plus 2." Or just: E minus N plus Two = paths in queue.

Master Revision – Lectures 34–38

Topic	Key Takeaway
Portability	Use ANSI standards; text for data exchange; avoid bit fields
Exception Safety	Basic (no leaks) → Strong (atomic) → No-throw
Verification	Meets specifications? "Built product right?"
Validation	Meets user expectations? "Built right product?"
Defect	Any variance from desired product attribute
Testing Objective	Find bugs (show presence); cannot prove absence
Testing Phases	Unit→Module→Subsystem→System→Acceptance→Alpha→Beta
Black Box	Tests required behavior; no internal knowledge
White Box	Tests internal structure; code paths and branches
Equivalence Partitioning	Group inputs into classes with same testing behavior
Flow Graph	Directed graph of program control flow
Statement Coverage	Weakest; all statements executed at least once
Branch Coverage	All branches tested
Path Coverage	All paths tested; often infeasible
Cyclomatic Complexity	$V(G) = E - N + 2$; upper bound on test cases needed

Lecture 40: Unit Testing & Defect Removal Efficiency

Definition: Unit Testing

Testing individual software components (procedures, functions, classes) in ISOLATION from other components. Equivalent to chip-level testing in hardware.

"Build in self-testing: each unit should be testable independently." Testing philosophy.

Unit Testing Principles

- Developers test their own code units during implementation.
- Normal AND boundary inputs are tested against expected results.
- Unit testing is a great way to test an API.
- Use a test harness: an artificial environment that invokes routines being tested.

Benefits of Unit Testing

Type	Benefit	Explanation
Quantitative	Repeatable	Tests can be re-run to check no unintended side effects from code changes.
Quantitative	Bounded (Narrow focus)	Simplifies finding and fixing defects; smaller scope.
Quantitative	Cheaper	Find and fix defects EARLY before they cascade.
Qualitative	Assessment-oriented	Writing tests forces thinking about design (cohesion, coupling).
Qualitative	Confidence-building	Know what works early; easier to change when you can retest.

Testing Against the Contract

Unit tests ensure a given unit HONORS ITS CONTRACT. Example – Square Root:

```
result = squareRoot(argument); assert(abs(result * result - argument) < epsilon); // Test cases
from this contract: // 1. Pass NEGATIVE argument → should be REJECTED // 2. Pass ZERO → accepted
(boundary value) // 3. Pass valid positive values → |result^2 - argument| < epsilon
```

■ Exam Tip

Design tests BEFORE implementing the code. This forces you to think about the interface and boundary conditions BEFORE you write the implementation.

Defect Removal Efficiency

Definition: Defect Removal Efficiency (DRE)

The ability to remove defects from an application. Measured as the percentage of defects caught by a quality technique.

Key data from 1500 project analysis:

Technique Combination	Median Efficiency
Testing alone	53%
Testing + Quality Assurance	65%
Testing + Code Inspection	75%
Testing + Design Inspection	80%

Design Inspect + QA + Testing	95%
All four combined	99.9%

Conclusion: Testing ALONE is insufficient. Combining with inspections dramatically improves defect removal.

Defect Origination Points

- Requirements
- Design
- Coding
- User documentation
- Testing itself (bad fixes introduce new bugs)
- Change requests at maintenance phase

■ Important Note

Chaotic Zone: When defects are not fixed at their origination point, they accumulate at testing and maintenance phases, creating chaos. Fix defects as close to their origin as possible.

Short Questions

Q: What is unit testing?

Testing individual software components (functions, classes, modules) in isolation. Equivalent to chip-level testing in hardware. Developers test their own code during implementation.

Q: What is a "test harness" in unit testing?

A test harness is an artificial testing environment that invokes the routines of the module being tested and checks the results against known values or previous runs (regression testing).

Q: What does it mean to test against the contract?

A contract specifies what the function promises to do. Testing against the contract means writing test cases that verify the function fulfills that promise, including boundary cases and invalid inputs.

Q: What is Defect Removal Efficiency?

DRE is the percentage of defects that a quality assurance technique successfully identifies and removes. Testing alone achieves ~53% median. All four techniques (design inspection, code inspection, QA, testing) combined achieve 99.9%.

Q: What is the "Chaotic Zone" in defect management?

When defects are not discovered and fixed at their point of origination, they pile up at the testing and maintenance phases. This creates chaos because fixing requirements or design defects late requires changing all dependent code.

MCQs

Q106. Unit testing is equivalent to what in hardware?

- (A) System testing
- (B) Integration testing
- ✓ (C) Chip-level testing
- (D) Acceptance testing

Explanation: Unit test = chip-level test (each component independently).

Q107. Testing alone achieves what median defect removal efficiency?

- (A) 90%
- (B) 75%
- ✓ (C) 53%
- (D) 99%

Explanation: Testing alone: ~53% median efficiency.

Q108. Design tests BEFORE implementing code is called:

- (A) Post-hoc testing
- ✓ (B) Contract-driven testing / TDD
- (C) Regression testing
- (D) Black box testing

Explanation: Test-first = testing against the contract / Test-Driven Development.

Q109. The "Chaotic Zone" refers to:

- (A) Automated test failures
- ✓ (B) Accumulated unfixed defects overwhelming testing/maintenance
- (C) Random testing
- (D) Performance bottlenecks

Explanation: Chaotic zone = piled-up defects destabilizing testing/maintenance.

Q110. Which combination achieves 99.9% defect removal efficiency?

- (A) Testing alone
- (B) Testing + design inspection
- (C) Testing + QA
- ✓ (D) All four (design inspection, code inspection, QA, testing)

Explanation: All four techniques combined = 99.9%.

■ Memory Trick

DRE increases with each technique added

Testing alone = 53%. Add ONE technique = 65-80%. Add ALL = 99.9%. Each layer of quality assurance catches what the previous missed. NEVER rely on testing alone!

Lecture 41: Inspection vs. Testing, Static Analyzers

1. Inspection vs. Testing – Key Differences

Aspect	Inspection	Testing
Execution required?	NO – does not require program execution	YES – requires program execution
When applicable?	BEFORE implementation (requirements, design, code)	AFTER implementation only
What it checks?	Conformance with specification	Actual runtime behavior
Multiple defects?	Many defects found in single session	One defect may mask another
Non-functional?	Cannot check performance, usability	Can check performance, usability
Real requirements?	Cannot verify against customer's real needs	Can validate against real behavior

Conclusion: Inspections and testing are **COMPLEMENTARY**, not competing. **BOTH** should be used.

2. Inspection Preconditions

- A precise specification must be available.
- Team members must be familiar with organization standards.
- Syntactically correct code must be available.
- Inspectors should prepare a checklist beforehand.

3. Inspection Checklists

Checklists contain common errors for inspectors to look for. They are programming language dependent:

Fault Class	Inspection Checks
Data faults	Are all variables initialized before use? Are all constants named? Correct array bounds (size vs size-1)?
Control faults	Is each conditional correct? Is each loop certain to terminate? Are compound statements properly bracketed?
Input/Output faults	Are all input variables used? Are output variables assigned before output?
Interface faults	Correct number of parameters? Types match? Parameters in right order?
Storage management	All links correctly assigned in linked structures? Space allocated and deallocated correctly?
Exception management	Have all possible error conditions been considered?

4. Static Analyzers

Definition: Static Analyzer

Software tools that parse program text and automatically discover potentially erroneous conditions **WITHOUT** executing the program. They supplement but do **NOT** replace inspections.

Fault Class	Static Analysis Checks
Data faults	Variable used before initialization; declared but never used; assigned twice without intervening use; array
Control faults	Unreachable code; unconditional branches into loops.
Input/Output	Variable output twice with no intervening assignment.
Storage Management	Unassigned pointers; pointer arithmetic.

Short Questions

Q: List two advantages of inspection over testing.

1) No program execution required – can be applied before coding. 2) Many defects found in a single inspection session (testing one defect may mask another).

Q: List two limitations of inspection compared to testing.

1) Cannot check non-functional requirements (performance, usability). 2) Cannot verify conformance with customer's real requirements (only with written spec).

Q: What is a static analyzer?

A software tool that analyzes program source code WITHOUT executing it. It detects potentially erroneous conditions like uninitialized variables, unreachable code, and undeclared variables.

Q: What are inspection preconditions?

1) Precise specification available. 2) Team familiar with standards. 3) Syntactically correct code available. 4) Inspectors have prepared checklists.

Q: Why are inspection checklists programming language dependent?

Different languages have different common error patterns. For example, weakly-typed languages have more type-related errors; C has pointer errors. Checklists must cover major constructs specific to the language being inspected.

MCQs

Q111. Inspections can be applied:

- (A) Only after coding
- ✓ (B) Before implementation (requirements, design, code)
- (C) Only during testing
- (D) Only to documentation

Explanation: Inspections: requirements, design, code – all before/after implementation.

Q112. A static analyzer detects errors:

- (A) At runtime
- ✓ (B) Without executing the program
- (C) During testing
- (D) After deployment

Explanation: Static analysis = no execution needed.

Q113. Which is NOT checked by inspections?

- (A) Variable initialization
- (B) Loop termination
- ✓ (C) Runtime performance
- (D) Interface parameters

Explanation: Inspections cannot check runtime properties like performance.

Q114. In testing, one defect may mask another. In inspections:

- (A) Same problem exists
- ✓ (B) Many defects found in one session
- (C) No defects are found
- (D) Only one defect found

Explanation: Inspection finds many defects in one review session.

Q115. Static analyzers supplement inspections; they are:

(A) A complete replacement

✓ (B) A supplement (not replacement)

(C) Faster than inspections

(D) Less effective than testing

Explanation: Static analyzers supplement but do not replace inspections.

■ Memory Trick

INSPECTION = pre-execution check | TESTING = post-execution check

Inspection is like a DOCTOR checking X-ray BEFORE surgery. Testing is like monitoring DURING surgery. Both needed!

Lecture 42: Debugging – History, Art & Science

Definition: Bug

"Things the software does that it is not supposed to do, or things the software doesn't do that it is supposed to."
Includes crashes, incorrect output, garbled displays.

Brief History of Debugging

- Admiral Grace Hopper coined the term "bug" – first bug was an actual MOTH in the Mark II computer relay (1940s).
- "From that point on, if her team was not producing numbers, they claimed to be debugging the system."
- Early debugging: data dumps, output devices (printers, lights).
- Next step: command-line debuggers.
- Modern: integrated debugging in IDEs.

Importance of Debugging

- Maintenance = 2/3 of total software lifecycle cost.
- 20% of maintenance cost is from defects found AFTER installation.
- AT&T; 1990: 9-hour US-wide telephone outage due to a single software bug (break statement in wrong scope).
- AT&T; 1998: 26-hour failure in frame-relay network from software upgrade bug.

The AT&T; 1990 bug: A break statement at line 7 was intended to break the if-then clause. Instead it broke out of the switch statement, causing incorrect execution flow. A single misunderstood break caused 9 hours of nationwide outage.

■ Important Note

AT&T; conclusion: "We believe our processes are solid. We will use this experience to further improve our procedures." Even rigorous testing can miss deeply nested logic errors.

Art and Science of Debugging

- Debugging is taken as art but is actually a SCIENTIFIC PROCESS.
- Developers build bias ("personal impression") about their own code and miss obvious errors.
- "Another pair of eyes" is essential – start with your own heuristics, ask for help if stuck.
- Bulletin Board Example: `while (i=0; i<10; i++)` – obvious for loop written as while!

■ Exam Tip

Scientific debugging approach: methodically check and verify code step by step rather than randomly guessing. Develop heuristics from experience.

Short Questions

Q: Who coined the term "bug" and how?

Admiral Grace Hopper coined "bug" when an actual moth flew into the Mark II computer's relay, causing a malfunction. The moth was taped into the logbook and the team said they were "debugging" the system.

Q: What caused the AT&T; 1990 9-hour outage?

A break statement in line 7 was inside an if-else block within a switch statement. The programmer intended it to break the if-clause. Instead it broke out of the switch, causing incorrect execution flow when a network node recovered.

Q: Why do developers miss bugs in their own code?

Developers build a personal bias/impression about their code. They see what they intended, not what is there. This personal bias prevents them from spotting obvious errors. Another person (another pair of eyes) breaks this bias.

Q: What does the Bulletin Board example demonstrate?

The code while (i=0; i<10; i++) was placed on a bulletin board. Almost nobody spotted the obvious syntax error: it uses a for loop's syntax inside a while loop declaration. This shows how developers miss obvious errors.

Q: What percentage of maintenance cost is attributed to defects found after installation?

20% of maintenance cost (which is itself 2/3 of total lifecycle cost). So roughly 13% of total software lifecycle cost goes to post-installation defect fixing.

MCQs**Q116. The term "bug" was coined by:**

- (A) Alan Turing
- ✓ (B) Admiral Grace Hopper
- (C) Edsger Dijkstra
- (D) Bill Gates

Explanation: Grace Hopper: moth in Mark II → "debugging".

Q117. The AT&T; 1990 outage lasted:

- (A) 1 hour
- ✓ (B) 9 hours
- (C) 26 hours
- (D) 3 days

Explanation: AT&T; 1990: 9-hour US-wide telephone outage.

Q118. Why should developers NOT debug their own code alone?

- (A) They are too busy
- ✓ (B) Personal bias makes them miss obvious errors
- (C) Debugging is too hard
- (D) It is not their job

Explanation: Personal bias = developer sees intent, not actual code.

Q119. The Bulletin Board example had what type of error?

- (A) Logic error
- ✓ (B) Syntax error (while with for syntax)
- (C) Runtime error
- (D) Memory leak

Explanation: while loop written with for loop syntax.

Q120. Debugging should be treated as:

- (A) Art only
- (B) Random guessing
- ✓ (C) A scientific process
- (D) A developer's hobby

Explanation: Debugging = scientific process with systematic heuristics.

■ Memory Trick

BUG = Behavior Unexpectedly Gone wrong

Remember: Grace Hopper found a real bug (moth) in a real computer. Modern bugs are metaphorical but equally real in their damage!

Lecture 43: Bug Classes – Memory Leaks, Logical, Loop, Pointer, Boolean

1. Memory and Resource Leaks

Definition: Memory Leak

Memory is allocated from OS or memory pool but NEVER DEALLOCATED when finished. The allocated memory remains unavailable for the rest of the program's life.

Symptoms: System slowdowns; random crashes over long periods.

Example – Memory allocated but not freed on error path:

```
char *buffer = new char[kMaxBufferSize+1]; if (IsError(nCondition)) { return FailureCode; // BUG:
buffer NOT freed! } delete buffer; // Only reached on success path
```

Fix: Always free memory in ALL exit paths, or use RAI (Resource Acquisition Is Initialization) with smart pointers.

2. Logical Errors

Definition: Logical Error

Code is syntactically correct but does NOT do what is expected. Program doesn't crash but behavior is wrong or opposite to expected.

Example – AND/OR confusion:

```
if ((input >= 1 && input <= 10) && (input >= 15 && input <= 20)) // No number can be between 1-10
AND between 15-20 simultaneously! // Should be: || (OR) not && (AND)
```

3. Coding Errors

Definition: Coding Error

Simple coding mistakes: failure to check error returns, failure to handle valid conditions, incorrect parameter passing, invalid return type coercion.

Example – convertToString doesn't handle case 0 and doesn't initialize output variables.

4. Memory Over-runs

Definition: Memory Over-run

Using memory that doesn't belong to you. Caused by overstepping array bounds or copying a string too large for its buffer.

```
const kMaxEntries = 50; int gArray[kMaxEntries]; void ZeroArray(int *pArray) { for (int i=0; i<100;
++i) // BUG: 100 iterations, only 50 elements! pArray[i] = 0; }
```

Symptoms: Crashes after a routine is called; trace log analysis shows the problem in prior routines.

5. Loop Errors

Definition: Loop Error

Problems around loop constructs: infinite loops, off-by-one errors, improperly exited loops.

```
// Off-by-one error: int anArray[50]; i = 50; // BUG: valid indices are 0-49, starting at 50 is out
of bounds! while (i >= 0) { anArray[i] = 0; // writes to anArray[50] - out of bounds! i = i - 1; }
```

6. Pointer Errors

Definition: Pointer Error

Using pointers incorrectly: uninitialized pointers, deleted pointers still in use, invalid pointers pointing to wrong memory.

```
void cleanup(char *ptr) { SaveToDisk(ptr); delete ptr; // ptr deleted here! } void func() { char *s
= new char[80]; cleanup(s); delete s; // BUG: double delete! ptr already freed in cleanup. }
```

7. Boolean Bugs

Definition: Boolean Bug

Occurs because mathematical Boolean algebra has nothing to do with English words. "and" in English often means Boolean "or" and vice versa.

Counter-intuitive return value example: A function returns 0 for failure but the calling code checks if (Ret) { cout << "Error"; } – since 0 is false, error message never prints on failure!

■ Exam Tip

Boolean bug tip: Always make return values intuitive. Use named constants like SUCCESS=0, FAILURE=1 and document what the return means.

Summary of Bug Classes

Bug Class	Key Symptom	Root Cause
Memory Leak	Slowdown over time; random crashes	Memory not freed on all exit paths
Logical Error	Wrong results; opposite behavior	Wrong operator/logic (&& confusion)
Coding Error	Unexpected failures; unhandled cases	Missing case handlers; wrong params
Memory Over-run	Crash after specific routine	Array boundary exceeded
Loop Error	Infinite loop; off-by-one results	Wrong termination condition
Pointer Error	Crashes; unpredictable variable changes	Double delete; uninitialized pointer
Boolean Bug	Opposite behavior to expected	Counter-intuitive boolean return values

Short Questions

Q: What is a memory leak? Give example.

Memory is allocated but never freed. Example: buffer allocated before error check; on error, function returns early without freeing buffer. Memory stays allocated for rest of program life, causing slowdown/crashes.

Q: What is a logical error?

Syntactically correct code that doesn't do what is expected. Example: using && (AND) when || (OR) was intended. Program compiles and runs but gives wrong results.

Q: What is a memory over-run?

Writing to memory that doesn't belong to you – typically by going past array boundaries. Example: a 50-element array iterated 100 times, writing into adjacent memory.

Q: What is a pointer error? Give two types.

Using pointers incorrectly. Types: 1) Uninitialized pointer (points to random memory). 2) Double delete (freeing already-freed memory). 3) Invalid pointer (points to valid memory but wrong data).

Q: What is a Boolean bug? Give an example.

Boolean algebra doesn't match English. Example: "select at least one of these" should be OR (||) not AND (&&). Also: functions returning 0 for failure when calling code checks if (ret) meaning "if true", not if "0".

MCQs

Q121. A memory leak occurs when:

(A) Array bounds are exceeded

✓ (B) Allocated memory is never freed

(C) Pointer is uninitialized

(D) Loop runs forever

Explanation: Memory leak = allocated memory never returned to OS/pool.

Q122. which error is the code: `if(a>=1&&a;<=10 && a>=15&&a;<=20)?`

(A) Memory leak

✓ (B) Logical error

(C) Coding error

(D) Loop error

Explanation: Logical: no number can satisfy both conditions simultaneously.

Q123. An off-by-one loop error is a type of:

(A) Memory leak

(B) Pointer error

✓ (C) Loop error

(D) Boolean bug

Explanation: Off-by-one is a classic loop error.

Q124. Double delete of a pointer causes:

(A) Memory leak

✓ (B) Undefined behavior/crash

(C) Logic error

(D) No effect

Explanation: Double delete = undefined behavior, usually crashes.

Q125. Boolean bugs often occur because:

(A) Variables are wrong type

✓ (B) English "and" corresponds to Boolean "or" in many contexts

(C) Loop termination is wrong

(D) Pointers are null

Explanation: English "and" vs Boolean && often confused.

■ Memory Trick

7 Bug Classes = M-L-C-M-L-P-B

Memory leak, Logical, Coding, Memory overrun, Loop, Pointer, Boolean. Remember: "Many Lazy Coders Make Lousy Programs Badly."

Lecture 44: Holistic Debugging & Debugging Process

Definition: Holistic Approach

Emphasizing the importance of the whole and the interdependence of its parts. In debugging, focus on the ENTIRE APPLICATION SYSTEM, not just the symptom.

"You cannot treat the symptoms of a problem; you must focus on the application system as a whole."

Example: Program crashes in `SetX(int X) { mX = X; }` – this simple line cannot be wrong. Adding a print statement makes it disappear. The REAL bug is elsewhere – a pointer error in a DIFFERENT function is corrupting memory, making this line appear to fail.

The Debugging Process

- Start with user description of the problem (treat it suspiciously – get first-hand account).
- Write down EXACTLY what you are told.
- FILTER: identify what multiple reports have in common.
- Look for the COMMON ELEMENT across different bug reports.
- Use common element to identify likely root cause.

Three User Reports Example:

User	Reported Action
User 1	Set up Favorites list → scrolled → pressed Enter → moved to submenu → clicked → entered name "Irving" → clicked O
User 2	Programming menu → New menu option → Create Object → entered "HouseObject" → clicked OK → CRASH
User 3	New Project → clicked Gear icon → entered name "Rudolph" → clicked OK → CRASH

Common elements: ALL users used BOTH keyboard AND mouse for menu selections BEFORE clicking OK. Likely root: menu handler does not properly handle mixed keyboard+mouse entry.

Good Clues for Easy Bugs

Stack Trace:

- Source line numbers in stack trace are the single MOST USEFUL piece of debugging information.
- Values of arguments in stack trace are the second most useful.
- Look for improbable values (zero, very large, negative, non-alphabetic chars).
- Use debugger to display local and global variable values.

Non-Reproducible Bugs:

- Hardest bugs to fix – they appear "randomly".
- Randomness itself is information.
- Check: are ALL variables initialized? (random data in uninitialized vars affects output)
- Does bug disappear when debug code inserted? → likely memory allocation problem.
- Is crash site far from anything obviously wrong? → check for dangling pointers.

```
// Classic dangling pointer in function: char *msg(int n, char *s) { char buf[100]; // LOCAL
variable on stack! sprintf(buf, "error %d: %s\n", n, s); return buf; // BUG: returning pointer to
local (stack) variable! // buf is destroyed when function returns! }
```

Lecture 45: Summary & Complete Revision

Short Questions – Lectures 44–45

Q: What is the holistic approach to debugging?

Looking at the entire application system as a whole rather than just treating the reported symptom. The crash site is often NOT where the real bug is; the root cause may be in a different part of the code corrupting memory.

Q: How do you find common elements across multiple bug reports?

Filter out user-specific irrelevant details. Look for patterns: What action is common? What UI element? What sequence? The common element points to the likely root cause.

Q: What is a dangling pointer?

A pointer that points to memory that has already been freed or gone out of scope. Accessing a dangling pointer causes undefined behavior (crashes, wrong values).

Q: What makes non-reproducible bugs so difficult?

They appear randomly, making them hard to reproduce and isolate. They are often caused by uninitialized variables (random data), timing issues, or memory corruption that manifests differently each run.

Q: What is the most useful information in a stack trace?

Source line numbers are the single most useful piece. Then the values of arguments passed at each level. These tell you exactly where the crash occurred and what data was involved.

MCQs – Lectures 44–45

Q126. The holistic debugging approach focuses on:

- (A) The exact crash line
- ✓ (B) The entire application system
- (C) User reports only
- (D) Stack trace only

Explanation: Holistic = entire system, not just symptom site.

Q127. Non-reproducible bugs are often caused by:

- (A) Syntax errors
- ✓ (B) Uninitialized variables with random data
- (C) Missing return statements
- (D) Wrong algorithm

Explanation: Random data in uninitialized vars causes non-reproducible bugs.

Q128. Returning a pointer to a local variable causes:

- (A) Memory leak
- ✓ (B) Dangling pointer / undefined behavior
- (C) Compilation error
- (D) Logical error

Explanation: Local variable destroyed on return; pointer becomes dangling.

Q129. The most useful item in a stack trace is:

- (A) Function names only
- ✓ (B) Source line numbers
- (C) Return values

(D) Object names

Explanation: Source line numbers = most useful stack trace information.

Q130. When multiple users report the same crash, you should:

(A) Fix the first report only

✓ **(B) Find the COMMON element across all reports**

(C) Ignore the reports

(D) Ask each user to test differently

Explanation: Common element across reports = likely root cause.

■ Memory Trick

DEBUG = Don't Expect Bug in Given location

The crash site is often NOT where the real bug is. Always trace backwards to find the root cause.

Master Revision – Lectures 39–45

Topic	Key Takeaway
Statement Coverage	Weakest; all statements executed once
Branch Coverage	All branches (true/false) tested
Path Coverage	All paths; often infeasible
Cyclomatic Complexity	$V(G)=E-N+2$; upper bound on test cases
Infeasible Path	Flow graph path that can never execute
Unit Testing	Components tested in isolation; chip-level equivalent
DRE	Testing alone = 53%; all 4 techniques = 99.9%
Chaotic Zone	Unfixed early defects overwhelm testing/maintenance
Inspections	No execution; pre-implementation; finds many defects at once
Static Analyzers	Supplement inspections; no execution needed
Bug = Defect	What software does (or doesn't) that it shouldn't (or should)
Memory Leak	Allocated memory never freed
Logical Error	Correct syntax; wrong logic (&&/ confusion)
Memory Over-run	Array boundary exceeded
Loop Error	Infinite loop or off-by-one
Pointer Error	Dangling/uninitialized/double-delete
Boolean Bug	Unintuitive boolean return values
Holistic Debugging	Focus on whole system, not just crash site
Stack Trace	Source line numbers = most useful debug info

Lecture FINAL: Complete Final Revision – Definitions, MCQs, Q&A;, Exam Guide

100 Most Important Definitions

1. **Gen-Spec Structure:** IS-A inheritance hierarchy; child is a specialized kind of parent.
2. **Whole-Part Structure:** HAS-A relationship; whole is composed of part objects.
3. **Sequence Diagram:** UML diagram showing message exchanges between objects over time (vertical = time).
4. **Lifeline:** Dashed vertical line in Sequence Diagram representing object's existence during interaction.
5. **Activation Box:** Thin rectangle on lifeline showing period of active message processing.
6. **Synchronous Message:** Caller BLOCKS and waits for called method to complete.
7. **Asynchronous Message:** Caller does NOT wait; continues immediately. Used in multithreading.
8. **Collaboration Diagram:** UML diagram showing object interactions with numbered messages (organization-based view).
9. **Coupling:** Degree of dependency between modules/classes. LOW coupling is desirable.
10. **Cohesion:** Degree elements within a class belong together. HIGH cohesion is desirable.
11. **Software Architecture:** Structure(s) of a system: components + externally visible properties + relationships.
12. **Sub-system:** A system in its own right; operates independently of other sub-systems.
13. **Module:** System component providing services; not considered a separate system.
14. **Krutchen 4+1 Model:** Architecture model with 5 views: Logical, Process, Physical, Development, Scenarios.
15. **Logical View:** Object model of the design (classes and relationships).
16. **Process View:** Concurrency and synchronization aspects.
17. **Physical View:** Hardware mapping and distribution.
18. **Development View:** Static organization: files and directories.
19. **Repository Model:** Central shared database accessed by all sub-systems.
20. **Client-Server Model:** Clients use services from servers connected over network.
21. **Thin Client:** All processing at server; client only handles display.
22. **Fat Client:** Processing distributed to client machine.
23. **Zero Install:** No client installation needed; all processing server-side (web apps).
24. **N-Tier Architecture:** Multiple server tiers for scalability (web+app+DB servers).
25. **Pipes and Filters:** Independent filter components connected by pipes; data flow architecture.
26. **Reference Architecture:** Standard non-physical model (e.g., OSI) for protocol definition.
27. **Layered Architecture:** Multiple layers; inner layers closer to hardware; interfaces between layers enable portability.
28. **Horizontal Partitioning:** Separate branches per major function; control modules coordinate.
29. **Vertical Partitioning:** Decision-makers at top; worker modules at bottom.
30. **Design Pattern:** Reusable solution to a recurring design problem in a specific context (GoF).
31. **Creational Patterns:** Patterns about HOW objects are created (Singleton, Factory, Abstract Factory).
32. **Structural Patterns:** Patterns about HOW objects/classes compose into larger structures (Facade, Adapter).
33. **Behavioral Patterns:** Patterns about HOW objects communicate (Observer, Strategy, Command).
34. **Observer Pattern:** One-to-many dependency; Subject notifies all Observers on state change.
35. **Singleton Pattern:** Ensures exactly one instance of a class with a global access point.
36. **Facade Pattern:** Provides a simplified unified interface to a complex subsystem.
37. **Self-Documenting Code:** Code explaining itself without comments through naming and structure.
38. **Hungarian Notation:** Variable naming convention including type/scope information in the name.

39. **CamelCase:** Compound words joined with each word capitalized (Java's official convention).
40. **Magic Number:** Unexplained numeric literal in code; should be replaced with named constant.
41. **Include Guard:** `#ifndef/#define/#endif` construction preventing multiple header inclusion.
42. **Short-Circuit Evaluation:** `||` and `&&` stop evaluation when result can be determined.
43. **Side Effect:** Function modifying parameters or external variables besides its return value.
44. **80/20 Rule:** 80% of execution time in 20% of code; profile first, optimize hot spots.
45. **Portability:** Ability to move software to different platforms with minimum effort.
46. **Endianness:** Byte order in multi-byte storage: little-endian (Intel) vs big-endian (Motorola).
47. **Exception Handling:** Mechanism separating error-handling code from normal code via `try/catch/throw`.
48. **Basic Guarantee:** No memory leaks; temporaries destroyed properly.
49. **Strong Guarantee:** Full commit-or-rollback: either all or no side effects on exception.
50. **No-throw Guarantee:** Function will never throw an exception.
51. **Exception-Safe Function:** Might throw but has NO side effects if it does.
52. **Exception-Neutral Function:** Propagates all exceptions to the caller.
53. **Software Verification:** "Built product right?" – checking against specifications.
54. **Software Validation:** "Built right product?" – checking against user expectations.
55. **Defect:** Any variance from desired product attribute causing customer dissatisfaction.
56. **Software Testing:** Executing software on test data and comparing output against expected behavior.
57. **Test Case:** Input/output specification + function under test + steps + expected results.
58. **Test Data:** Actual input values used to execute test cases.
59. **Black Box Testing:** Testing required behavior without knowledge of internal implementation.
60. **White Box Testing:** Testing based on internal code structure (paths, branches, statements).
61. **Unit Testing:** Testing individual components (functions, classes) in isolation.
62. **Module Testing:** Testing a collection of dependent components independently.
63. **Sub-system Testing:** Testing collections of modules; finds interface problems.
64. **System Testing:** Testing the complete integrated system as a whole.
65. **Acceptance Testing:** Validation against user expectations; usually at client site.
66. **Alpha Testing:** In-house acceptance testing for products before beta.
67. **Beta Testing:** Field testing with potential customers before general release.
68. **Equivalence Class:** Group of test inputs with same defect-discovery behavior.
69. **Equivalence Partitioning:** Dividing input domain into equivalent classes for test case selection.
70. **Flow Graph:** Directed graph representing program control flow (nodes=statements, edges=flow).
71. **Statement Coverage:** Every statement executed at least once. Weakest coverage.
72. **Branch Coverage:** Every decision branch (true/false) tested at least once.
73. **Path Coverage:** Every possible execution path tested. Often infeasible.
74. **Cyclomatic Complexity:** $V(G) = E - N + 2$. Quantitative measure of logical complexity. Upper bound on test cases.
75. **Independent Path:** Any path through program introducing at least one new processing statement or condition.
76. **Infeasible Path:** Path existing in flow graph but never traversable for any input data.
77. **Test Harness:** Artificial environment that invokes and tests routines; supports regression testing.
78. **Defect Removal Efficiency (DRE):** Percentage of defects successfully identified and removed by a technique.
79. **Chaotic Zone:** Accumulated unfixed defects overwhelming testing/maintenance phases.
80. **Inspection:** Team review of software artifacts WITHOUT executing the program.
81. **Static Analyzer:** Tool parsing source code to find potential errors without execution.
82. **Bug:** What software does (or doesn't do) that it shouldn't (or should do).

- 83. **Memory Leak:** Allocated memory never deallocated; causes slowdown and random crashes.
- 84. **Logical Error:** Syntactically correct code with wrong logic (e.g., && vs ||).
- 85. **Memory Over-run:** Writing to memory beyond allocated bounds (array boundary exceeded).
- 86. **Loop Error:** Infinite loop, off-by-one, or improperly exited loop.
- 87. **Pointer Error:** Dangling pointer, uninitialized pointer, or double-delete.
- 88. **Boolean Bug:** Unintuitive boolean return value causing opposite behavior to expected.
- 89. **Holistic Debugging:** Focusing on entire application system, not just crash symptom site.
- 90. **Stack Trace:** Record of function call sequence at crash time; source line numbers most useful.
- 91. **Non-reproducible Bug:** Bug appearing randomly, often from uninitialized variables or race conditions.
- 92. **Dangling Pointer:** Pointer to memory already freed or out of scope; causes undefined behavior.
- 93. **Anti-pattern:** Common design mistake documented to help developers avoid it.
- 94. **Program Family:** Set of programs where it is profitable to consider them as a group.
- 95. **Work Breakdown Structure (WBS):** Division of project work into components; based on module structure.
- 96. **Self-call (Sequence Diagram):** Message an object sends to itself.
- 97. **Encapsulation:** Hiding internal details; only exposing interface. Core OOP principle.
- 98. **Delegation:** Passing responsibility to another object (e.g., Room.doYouNeedHeat()).
- 99. **Protocol (Class Interface):** Set of all public messages a class can respond to. Minimize protocol size.

50 Most Important Long Questions & Answers

Q1: Explain Gen-Spec and Whole-Part structures with examples from Connie's Convenience Store.

Answer: Gen-Spec (IS-A): Payment → Cash Payment / Authorized Payment → Check / Card. Promotional Price → Price. Return → Sale. These reduce duplication by placing common attributes in the parent class. Whole-Part (HAS-A): Store contains Cashiers, Registers, Items. Register contains Cash Drawer. Sale contains Sale Line Items. These model composition/aggregation. Both structures drive the responsibility assignment in the object model.

Q2: What are Sequence Diagrams? Explain their notation in detail.

Answer: Sequence Diagrams show how objects interact over time. Notation: Objects on X-axis, time on Y-axis. Boxes=objects. Dashed vertical lines=lifelines. Thin rectangles on lifelines=activation boxes (active processing period). Horizontal arrows between lifelines=messages (solid=synchronous, half-arrow=asynchronous). Conditions in square brackets [condition] before message. Return values shown as dashed arrow back. Self-calls allowed. Object naming: [instanceName][:className].

Q3: Compare Sequence Diagrams and Collaboration Diagrams.

Answer: Both show object interactions but differ in perspective. Sequence: time-based, vertical axis=time, message order shown by vertical position, best for seeing flow of time. Collaboration: organization-based, messages numbered (1,2,3), best for seeing static object connections and control flow. Sequence Diagrams are usually easier to read for time order. Collaboration Diagrams can become complex very quickly.

Q4: Why is software architecture important? Explain three reasons.

Answer: 1) Mutual Communication: provides a common high-level abstraction for all stakeholders (managers, developers, customers) to discuss and form consensus. 2) Early Design Decisions: architecture embodies earliest design decisions that carry disproportionate long-term weight; it determines WBS, team structure, schedules. 3) Reusable Abstraction: architecture model transferable across similar systems; enables product line development.

Q5: Explain Krutchen's 4+1 Architectural View Model in detail.

Answer: Five views: 1) Logical View (object model, classes/relationships; for analysts and end-users). 2) Process View (concurrency, synchronization; for system integrators). 3) Physical View (hardware mapping, distribution; for hardware engineers). 4) Development View (files, directories; for programmers). 5) Use Case/Scenario View (validates other views; ties them together; for all stakeholders). Scenarios are the "+1" that integrate all four main views.

Q6: Compare Repository Model and Client-Server Architecture.

Answer: Repository: central shared DB, all subsystems access same data, efficient for large sharing, but subsystems must agree on one data model and evolution is expensive. Client-Server: distributed, clients call services from servers over network, makes effective use of networked systems, easier to add servers, but no standard for data sharing and redundant management per server. Repository = centralized data; Client-Server = distributed processing.

Q7: Explain Design Patterns. Describe the GoF format for documenting patterns.

Answer: Design patterns are reusable solutions to recurring design problems. GoF format (10 elements): 1) Name (meaningful idiom), 2) Problem (statement of intent), 3) Context (preconditions), 4) Forces (constraints and interactions), 5) Solution (structural description), 6) Examples, 7) Resulting Context (state after; consequences), 8) Rationale (justification), 9) Related Patterns, 10) Known Uses (proof of proven solution).

Q8: Explain Observer, Singleton, and Facade patterns with examples.

Answer: Observer: One-to-many; Subject notifies all Observers on state change. Example: Spreadsheet (Subject) + Bar Chart (Observer) – when data changes, chart updates. Participants: Subject, Observer, ConcreteSubject, ConcreteObserver. Singleton: Exactly one instance, global access. Implementation: protected constructor, static Instance() method, static _instance variable. Facade: Simplified interface to complex subsystem. Example: Compiler API hiding scanner/parser/codegen complexity. Reduces client-subsystem dependencies.

Q9: Explain self-documenting code with examples of naming conventions.

Answer: Self-documenting code explains itself without comments. Key attributes: 1) Function size ≤ 20 lines. 2) Meaningful identifiers (if(AllocFlag==NEW_NUMBER) vs if(x==0)). 3) Hungarian Notation (type info in name). 4) CamelCase for compound words. Conventions: types=PascalCase (Line), variables=camelCase (line), constants=ALL_CAPS (MAX_SIZE), methods=verb+camelCase (getName()), booleans=is/has/can prefix (isSet), private=underscore suffix (length_).

Q10: Explain exception handling levels of safety with the salary function example.

Answer: Three levels: 1) Basic: no leaks, temporaries cleaned up. 2) Strong: full commit-or-rollback. 3) No-throw: never throws. EvaluateSalaryAndReturnName example: Has 23 execution paths (3 normal, 20 exceptional). Original fails strong guarantee: partial messages may be emitted, or message emitted on function failure. Attempt 3 uses auto_ptr for non-throwing return (transfer of ownership), ensuring both side-effects complete or neither does.

Q11: Explain software verification vs validation. What is a defect?

Answer: Verification: "Built product right?" – conformance to specifications. Validation: "Built right product?" – conformance to user expectations. A system can be verified but not validated (spec doesn't capture real needs). Defect: any variance from desired product attribute causing customer dissatisfaction. Can be in specifications, design, code, documentation. Kernighan: "Death, taxes, and bugs are the only certainties."

Q12: What is the objective of software testing? What are its limitations?

Answer: Objective: expose latent defects before software deployment. Testing CANNOT prove absence of defects; only presence. "A successful test is one that finds an error." Limitations: 1) Input domain too large for exhaustive testing. 2) Too many paths through program. 3) Formally proving correctness requires mathematical proof, not tests. Example: isStringsEqual passes all test cases but still fails for same-length strings ending with same character.

Q13: Explain black box and white box testing with examples.

Answer: Black Box: treats system as black box; tests REQUIRED BEHAVIOR only; no knowledge of internals; from user's perspective. Also called functional testing. White Box: examines INTERNAL STRUCTURE; code design, paths, branches, statements; from developer's perspective. Also called structural testing. Black box finds missing/incorrect functionality. White box finds dead code, uncovered paths, logic errors.

Q14: Explain equivalence partitioning with a string matching example.

Answer: Equivalence partitioning divides input domain into classes where members have same defect-discovery behavior. For string matching: Equal partitions: all lowercase ("cat"/"cat"), all uppercase, mixed case, numeric, with blanks, null. Unequal partitions: different length, same length, case difference, one null. Test one from each class. Boundary conditions: empty string. Guidelines: organize classes, define boundaries, include invalid inputs.

Q15: Explain cyclomatic complexity with the bubble sort example.

Answer: Cyclomatic complexity $V(G) = E - N + 2$ where E =edges, N =nodes. It gives the upper bound on independent paths needed to test all statements and branches at least once. Bubble sort flow graph: 6 nodes, 8 edges $\rightarrow V(G)=8-6+2=4$. Four paths: 1) 1-6 (never enters while), 2) 1-2-3-4-5-1-6 (while once, swap), 3) 1-2-4-5-1-6 (while once, no swap), 4) 1-2-4-2-3-4-5-6-1 (inner for loop). 4 test cases cover all paths.

Q16: Explain unit testing. What are its quantitative and qualitative benefits?

Answer: Unit testing: testing individual components (functions, classes) in isolation during implementation. Against contract: test cases verify function honors its promise (e.g., squareRoot: test negative, zero, valid inputs). Quantitative: Repeatable (re-run to check side effects), Bounded (narrow focus), Cheaper (early defects). Qualitative: Assessment (forces design thinking), Confidence-building (know what works early).

Q17: Explain defect removal efficiency. What combination achieves best results?

Answer: DRE = % defects caught by technique. Testing alone: 53% median. Testing+QA: 65%. Testing+Code Inspection: 75%. Testing+Design Inspection: 80%. Design+QA+Testing: 95%. All four (design inspection + code inspection + QA + testing): 99.9%. Conclusion: Testing alone is insufficient. Combining inspection techniques with testing dramatically improves DRE.

Q18: Compare inspection and testing.

Answer: Inspection: no execution needed; applicable before implementation; finds many defects in one session; cannot check performance/usability; checks specification conformance. Testing: requires execution; after implementation; one defect may mask another; can check runtime behavior; validates against real requirements. They are COMPLEMENTARY, not competing. Both should be used throughout the development lifecycle.

Q19: Explain the different bug classes with examples.

Answer: Memory Leak: allocated memory never freed (buffer not freed on error path). Logical Error: $\&\&$ vs $\parallel$ confusion (number cannot be in 1-10 AND 15-20). Memory Over-run: array[50] iterated 100 times. Loop Error: off-by-one ($i=50$ for 0-49 array). Pointer Error: double delete (cleanup_function frees ptr, then func tries to free same ptr). Boolean Bug: function returns 0 for failure but caller checks if(Ret) expecting failure to be non-zero.

Q20: Explain the holistic debugging approach and debugging process.

Answer: Holistic: focus on the ENTIRE system, not just crash site. Symptom (crash in SetX) is not the bug; memory corruption elsewhere is. Process: 1) Get user description (treat suspiciously). 2) Write down exact reports. 3) Find COMMON ELEMENT across reports. 4) Use common element to identify root cause. Example: Three users crash on OK click, each using keyboard+mouse $\rightarrow$ menu handler with mixed keyboard+mouse entry is the bug.

Q21: Explain pipes and filters architecture with advantages.

Answer: Each processing step is a FILTER; connections between filters are PIPES. Key: filters are INDEPENDENT (don't know about each other). Batch sequential: linear sequence of filters. Non-batch: parallel/branching. Advantages: easy parallelism, filters are reusable, easy to add/remove filters. Disadvantage: shared data format required; no common state. Unix pipeline is the classic example: `ls | grep | sort | head`.

Q22: Explain the three-tier architecture with advantages over thin and fat client.

Answer: Three tiers: Client (presentation), Application Server (business logic), Database Server (data). Advantages over thin client: better performance (logic separated from DB). Advantages over fat client: simpler management (no client-side installation, no per-client updates). N-Tier generalizes this: more server types for different functions, increasing scalability.

Q23: Explain portability issues in C/C++ and how to address them.

Answer: Issues: 1) Data type sizes vary (int 2 or 4 bytes). 2) Char signedness unspecified. 3) Byte order (endianness) differs. 4) Evaluation order varies. 5) Bit field behavior platform-dependent. Solutions: Use ANSI/ISO standard C++. Use int (not char) for getchar(). Use text for data exchange. Avoid bit fields. Write function prototypes always. Use STL instead of vendor extensions.

Q24: Explain short-circuit evaluation with a real-world bug example.

Answer: In C/C++, && and || are short-circuit evaluated (left to right, stop when result known). Bug: `while(ptr->data < myData && ptr!=NULL)` – ptr is dereferenced BEFORE null check. If ptr is NULL, crash occurs. Fix: `while(ptr!=NULL && ptr->data < myData)` – guard FIRST. Short-circuit ensures ptr->data is only evaluated when ptr is known non-null.

Q25: What is the 80/20 rule in performance? How was it applied in the isspace example?

Answer: 80/20 Rule: 80% of execution time in 20% of code. Only optimize the hot spots (bottlenecks). isspace: profiling 10,000 calls found strchr (990k cycles) was bottleneck, called from strstr. Rewriting strstr eliminated strchr and strncmp. After optimization, strstr remained bottleneck at 99.8% of time. Further optimization using memcmp reduced strlen calls from 2 million to 652. Always PROFILE FIRST, then optimize.

Final 200 Practice MCQs – Quick Reference Sheet

Following are additional quick MCQs for rapid exam revision:

Q131. Which is NOT a benefit of Client-Server model?

(A) Low-cost hardware use

✓ (B) Central register of names

(C) Scalable

(D) Easy to add servers

Explanation: Client-Server LACKS a central register of names/services.

Q132. Asynchronous messages are used in:

(A) Single-threaded apps

✓ (B) Multi-threaded apps

(C) Database queries

(D) File access

Explanation: Async used in multi-threading/multi-processing.

Q133. The Perry-Wolfe formula for software architecture is:

(A) {Elements, Forms}

(B) {Components, Connectors}

✓ (C) {Elements, Forms, Rationale}

(D) {Classes, Objects}

Explanation: Perry-Wolfe: {Elements, Forms, Rationale/Constraints}.

Q134. Which view in Krutchen's model maps software to hardware?

(A) Logical View

(B) Process View

✓ (C) Physical View

(D) Development View

Explanation: Physical View = hardware mapping/distribution.

Q135. In the Observer pattern, ConcreteSubject:

(A) Defines update()

✓ (B) Stores state and notifies on change

(C) Provides global access

(D) Simplifies subsystem

Explanation: ConcreteSubject stores state and sends notifications.

Q136. Facade pattern is classified as:

(A) Creational

(B) Behavioral

✓ (C) Structural

(D) Functional

Explanation: Facade = Structural pattern.

Q137. Which naming convention does Java officially use?

(A) Hungarian Notation

(B) Snake_case

✓ (C) CamelCase

(D) ALL_CAPS

Explanation: Java uses CamelCase as official convention.

Q138. Returning a pointer to a local variable causes:

- (A) Memory leak
- ✓ (B) Dangling pointer
- (C) Correct behavior
- (D) Slower execution

Explanation: Local variable destroyed on function return; pointer dangles.

Q139. $V(G) = E - N + 2$ calculates:

- (A) Bug count
- (B) Code lines
- ✓ (C) Cyclomatic Complexity
- (D) Test data size

Explanation: Cyclomatic Complexity formula.

Q140. Which defect removal technique achieves 99.9% efficiency?

- (A) Testing alone
- (B) Inspections alone
- ✓ (C) All four techniques combined
- (D) Static analysis alone

Explanation: All four (design inspection+code inspection+QA+testing)=99.9%.

Q141. The "holistic" debugging approach means:

- (A) Debug only the crash line
- ✓ (B) Focus on the entire system
- (C) Ignore user reports
- (D) Use only stack traces

Explanation: Holistic = entire system perspective.

Q142. What does DRE stand for?

- (A) Defect Reporting Engine
- ✓ (B) Defect Removal Efficiency
- (C) Design Review Engine
- (D) Dynamic Runtime Error

Explanation: DRE = Defect Removal Efficiency.

Q143. Which pattern ensures exactly one instance of a class?

- (A) Observer
- (B) Facade
- (C) Factory
- ✓ (D) Singleton

Explanation: Singleton = one instance + global access.

Q144. Branch coverage is STRONGER than:

- (A) Path coverage
- ✓ (B) Statement coverage
- (C) Cyclomatic coverage
- (D) Loop coverage

Explanation: Branch > Statement coverage.

Q145. The AT&T; 1990 outage was caused by:

- (A) Virus

(B) Hardware failure

✓ (C) A break statement in wrong scope

(D) Memory leak

Explanation: break statement broke out of switch instead of if.

ONE-PAGE QUICK REVISION SHEET

QUICK REFERENCE FOR EXAM

Lectures 19-45 Core Concepts at a Glance

Gen-Spec	IS-A Whole-Part: HAS-A
Sequence Diagram	X=Objects, Y=Time, lifeline=dashed line, activation box=rectangle
Sync vs Async	Sync: caller BLOCKS Async: caller CONTINUES
Krutchen 4+1	Logical+Process+Physical+Development+Scenarios
Repository	Central shared DB Client-Server: distributed services
Thin/Fat	Thin: server side Fat: client side Zero Install: web-based
Design Pattern	C=Creational, S=Structural, B=Behavioral
Observer	Subject notifies many Observers Singleton: one instance Facade: simplified interface
Self-Docs Code	Function≤20 lines Type=PascalCase var=camelCase CONST=ALL_CAPS
Magic Numbers	Replace with named constants Comments: use //, minimize, indent
Short-Circuit	Guard FIRST in && && stops on false stops on true
80/20 Rule	Profile first Optimize hot spots (20% of code = 80% of time)
Exception Safety	Basic(no leak) → Strong(atomic) → No-throw
V&V;	Verification: meets spec Validation: meets user needs
Testing Phases	Unit→Module→Subsystem→System→Acceptance→Alpha→Beta
Black vs White	Black: required behavior (no internals) White: structural (internals)
Equiv. Partitioning	Group inputs with same defect behavior Test one per group
Coverage	Statement < Branch < Path (weakest to strongest)
V(G)	$E - N + 2$ Upper bound on test cases needed
DRE	Testing=53% +QA=65% +Code Inspect=75% All 4=99.9%
Inspection	No execution Pre-implementation Many defects in one session
Bug Classes	Memory Leak Logical Coding Overrun Loop Pointer Boolean
Holistic Debug	Crash site ≠ bug site Find common elements across user reports
Stack Trace	Source line numbers = most useful Check argument values

NIGHT BEFORE EXAM NOTES

- Gen-Spec = IS-A | Whole-Part = HAS-A
- Sequence Diagram: X-axis=Objects, Y-axis=Time. Sync=blocks. Async=doesn't block.
- 4+1 Views: Logical(object model), Process(concurrency), Physical(hardware), Development(files), Scenarios(use cases)
- Repository: central DB | Client-Server: distributed | Thin=server heavy | Fat=client heavy

- GoF Patterns: C=Creational, S=Structural, B=Behavioral. Observer=1→Many. Singleton=1. Facade=simplified interface.
- Self-documenting: ≤ 20 lines/function, meaningful names, named constants (no magic numbers)
- $V(G) = E - N + 2$ | Testing alone = 53% DRE | All 4 techniques = 99.9%
- Verification = meets spec | Validation = meets user needs | Testing can show presence, NOT absence of bugs
- Equivalence Partitioning: same-behavior groups | Black Box: required behavior | White Box: structure
- Bug Classes: Memory Leak (never freed), Logical (wrong $\&\&/||$), Overrun (array bounds), Loop (off-by-one), Pointer (double delete), Boolean (unintuitive return)
- Holistic debug: crash site $\neq$ bug site | Common elements across reports = root cause
- Exception Safety: Basic (no leak) → Strong (atomic/rollback) → No-throw

A+ GRADE TIPS

- ALWAYS define terms exactly before explaining – examiners want definitions first.
- Use examples: real examples from lectures (Connie's Store, Library, AT&T;) score extra points.
- For MCQs: eliminate obviously wrong options first, then use mnemonics.
- For long questions: structure your answer – definition → explanation → example → advantage/disadvantage.
- Know the $V(G)$ formula by heart: $E - N + 2$. Know bubble sort example: $E=8, N=6, V(G)=4$.
- Know DRE percentages: Testing alone=53%, All four=99.9%. Examiner loves these numbers.
- Know GoF pattern categories: Creational, Structural, Behavioral. Know one example of each.
- Know the difference between Verification and Validation – confusing these loses marks.
- For diagrams: draw neat, label every part, explain every label.
- Manage time: read all questions first, attempt easy ones first, come back to hard ones.

Best of Luck, Muhammad Haseeb Khan! You've Got This! ■

CS504 – Software Engineering I | VU | Lectures 19-45 | Complete Study Guide